Can NN Locality Bias Override Logical Hierarchy in Favor of Ordering Relations? *A Cross-Architectural Comparison of PLT and LTT Learning on CNN, RNN, and LSTM*

Stella Hong
LING 6300 S2026

1 Introduction

1.1 Background

Piecewise-Locally Testable (PLT) and Locally Threshold Testable (LTT) languages exhibit an exceptional set-theoretic inclusion relation within the Piecewise-Local hierarchy of formal language classes proposed in Rogers and Lambert (2019) and later expanded in Van Der Poel et al. (2024) to include the incomparable Tier-based classes in separate columns outside the sub-star-free portion of the hierarchy. The revised version, reproduced in Figure 1, additionally includes the SPL class as incorporated in the present study.

In this schematization, local and piecewise classes of stringsets within the regular-language boundary are organized along two dimensions: logical complexity and string-ordering relations. A pair of classes connected by set-theoretic inclusion, indicated by an arrow pointing from the superclass to the subclass, typically differs along only one of these dimensions while remaining constant along the other. For example, the ordering relation is held fixed while logical complexity varies in the inclusion relations LT⊆LTT, SL⊆LT, and SP⊆PT. Conversely, logical complexity is held fixed while the ordering relation varies in relations such as SL⊆TSL, PT⊆PLT, and LT⊆TLT.

These inclusion relations are formally explainable because one class can be viewed as a restricted case of the other. Along the logical-complexity dimension, strict classes are special cases of propositional classes: they are definable using only restricted forms such as conjunctions of negative literals (SL, SP) or disjunctions of positive literals (coSL, coSP), whereas propositional classes allow arbitrary Boolean combinations of atomic pattern predicates. Similarly, LTT extends LT by adding threshold-counting over local factors. Along the ordering-relation dimension, non-tier-based classes are special cases of tier-based classes in which the full alphabet Σ is projected onto the tier. Likewise, the piecewise-only class, PT, can be viewed as a special case of PLT in which the substring factor size is 1, i.e., $\mathrm{SPL}_{j,1}$ (Rogers and Lambert, 2019).

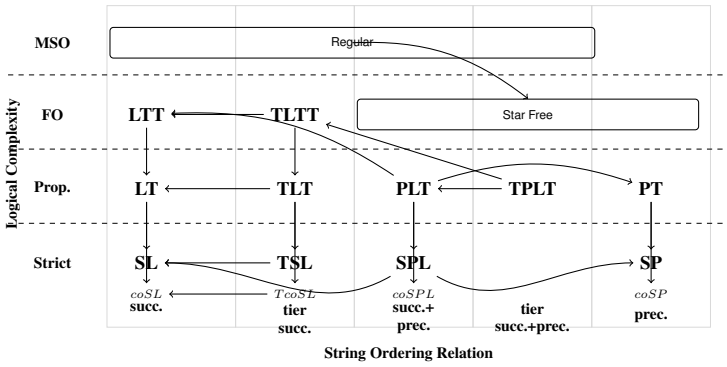

Figure 1: Revised Piecewise-Local hierarchy adapted from Van Der Poel et al. (2024) with SPL added.

Because these paired classes stand in a restricted-to-general relation, one might expect their relative learnability to be predictable from the set-inclusion hierarchy. The more general superclass permits a wider range of patterns and therefore defines a larger hypothesis space, which may increase cognitive or computational load and lead to greater learning difficulty. This prediction is largely borne out by the test accuracies obtained across different architectures on the MLReg benchmark (Van Der Poel et al., 2024). In most pairs, the subclass associated with the more restrictive grammar achieved higher accuracy, suggesting a lower learning burden (Table 1). Here, the direction of set inclusion is reversed in the accuracy ranking. The apparent exceptions are the last four rows: SP/coSP ⊆ PT, PT ⊆ PLT, and LTT ⊆ PLT.

Setting the former three cases aside for the moment, this paper focuses on the lattermost exception: the unexpected alignment between LTT and PLT. As can be seen in Figure 1, these classes occupy a unique position in which the logical hierarchy and the ordering-relation hierarchy conflict

with one another—a testament to the *orthogonality* of the two components.

On the one hand, in terms of logical complexity, PLT is a quantifier-free propositional logic, whereas LTT is a counting first-order logic, yielding the expectation that LTT $\subseteq$ PLT and that LTT should be more learnable than PLT.

On the other hand, along the dimension of ordering relation, LTT is based on the contiguous successor relation, whereas PLT draws on both successor and long-distance precedence relations in defining its factors. This leads to the opposite expectation, namely that PLT $\subseteq$ LTT and that LTT should be less learnable than PLT.

Set Relation	Accuracy
SL $\subseteq$ TSL	0.862 > 0.837
coSL $\subseteq$ TSL	0.854 > 0.837
TLT $\subseteq$ TLTT	0.8338 > 0.8289
LTT $\subseteq$ TLTT	0.8419 > 0.8289
LT $\subseteq$ TLT	0.8547 > 0.8338
SL $\subseteq$ LT	0.8620 > 0.8547
TSL $\subseteq$ TLT	0.837 > 0.833
LT $\subseteq$ LTT	0.8547 > 0.8419
LTT $\subseteq$ PLT	0.8419 < 0.865
PT $\subseteq$ PLT	0.816 < 0.865
SP $\subseteq$ PT	0.816 < 0.862
coSP $\subseteq$ PT	0.812 < 0.862

Table 1: Test accuracies on the MLReg benchmark (Van Der Poel et al., 2024), averaged across languages within each class, architectures, and test types.

The results in Table 1 support the former expectation, both in terms of the set-theoretic relation and the learnability inferred from test accuracy. Taken together, they suggest the precedence of logical complexity over ordering relation in defining formal language classes and their corresponding learnability. Nevertheless, this paper is motivated by the need to reexamine the learnability results—whose overall patterns are illustrated in Figure 2[1]—in light of the following two limitations.

First, Van Der Poel et al. (2024) evaluated performance using only a limited set of sequence-oriented architectures: simple RNN, LSTM, Transformer, and GRU. Because these models are particularly suited to sequential and long-distance dependency learning, their use may have obscured the observable effect of ordering relation, especially relative to more locality-biased architectures such as CNNs.

[1]Full statistics are reported in Table 7 in Appendix A.

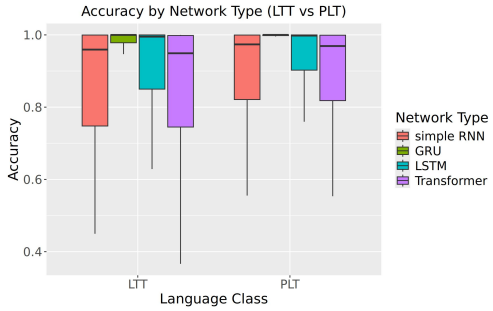

Figure 2: Bar plot of test accuracy on PLT and LTT across the architectures (Van Der Poel et al., 2024)

Second, non-negligible disparities exist in the number of parameters employed across models (Table 2), since these were selected through hyperparameter grid search with the goal of maximizing performance rather than providing an equal basis for cross-architectural comparison. This may introduce an additional source of imbalance in evaluation, as parameter count has been shown to affect comparative results, with LSTMs in particular being sensitive to such variation (Li and Zhou, 2026).

	Network Type			
Hyperparameters	**Simple RNN**	**GRU**	**LSTM**	**Transformer**
Embedding Dimension	32	32	256	256
Number of Feed Forward Layers	4	2	2	2
Approx. Parameter Count	19,020	23,244	670,000	870,000

Table 2: Hyperparameters selected from grid search in Van Der Poel et al. (2024), with approximate total parameter counts.

1.2 Objectives

Drawing on the aforementioned limitations, this work aims to revisit the learnability comparison between PLT and LTT languages using the same language test set employed in Van Der Poel et al. (2024), under two modifications.

First, in addition to sequential architectures (simple RNN and LSTM), a CNN model is introduced in order to test whether a more locality-biased learning algorithm increases sensitivity to ordering relations. Simple RNN is adopted as the baseline model, while LSTM is additionally included because it is more interpretable than the Transformer and less idiosyncratically high-performing than GRU models, as reported in Li and Zhou (2026); Van Der Poel et al. (2024). LSTM also presents an informative case: although it should in principle be more robust at learning non-local precedence relations, its performance on SP has been reported to

fall behind that of simple RNN Avcu et al. (2017).

Second, given the parameter imbalance noted in Table 2, we control parameter counts across architectures independently of their best-performing hyperparameter settings. This follows the procedure adopted in Li and Zhou (2026) to ensure a fairer basis for comparison. Since simple RNN is taken as the baseline, the settings of the other models (embedding dimension, number of feed-forward layers, and, in the case of CNN, number of channels) are adjusted to match its parameter count as closely as possible (Table 3).

	Network Type		
Hyperparameters	**CNN**	**Simple RNN**	**LSTM**
Embedding Dimension	32	32	32
Number of Feed Forward Layers	–	4	2
Architecture	22×3 Conv1D	–	–
Parameter Count	18,424	19,020	16,930

Table 3: Revised parameter-controlled model settings used in the present experiment.

The hypothesis we aim to test is whether the apparent precedence of logical complexity in determining set-theoretic relations *invariably* translates into learnability. More specifically, we ask whether the previously reported advantage of logically simpler classes reflects a genuine and architecture-invariant property of learnability, or whether it was partly an artifact of evaluating only sequential models under unequal parameter settings. If the effect remains stable across architectures, this would constitute strong evidence that ordering relations play a comparatively minor role relative to logical complexity in shaping learnability—at least in a computational setting. Accordingly, we formulate the following null and research hypotheses:

1. **H0 (Architecture Invariance).** The relative learnability ranking between LTT and PLT will remain stable across simple RNN, LSTM, and CNN, with no significant differences in their accuracies across architectures.

2. **H1 (Architecture Variance).** The relative learnability ranking between LTT and PLT will vary across architectures. In particular, models with stronger locality bias are expected to be more sensitive to ordering relations, thereby reducing the PLT advantage or potentially reversing it in favor of LTT.

   - **CNN** are expected to exhibit the strongest locality bias due to their sliding-window convolutional mechanism and position-invariant inductive bias, which favors substring-based pattern learning (Li and Zhou, 2026). They are therefore predicted to show the smallest PLT advantage, or possibly an LTT advantage.
   - **LSTM** The position of LSTM is less clear: although designed to model longer dependencies, prior work reports weaker performance on subsequence dependencies (Avcu et al., 2017).
   - **Simple RNN** serves as the sequential baseline against which these contrasts are evaluated.

.

2 PLT and LTT Languages

This section provides formal definitions of the PLT and LTT classes and explains how their defining traits are reflected in the MLReg dataset used in the present study. It also gives further details on the subset employed here and the four distinct test sets against which learning results were evaluated.

2.1 Locally Threshold Testable (LTT)

Defining classes of formal languages is best approached by building upward from the lowest level of logical complexity while holding the ordering relation constant. For Locally Threshold Testable (LTT) languages, we begin with the strict logical classes based solely on the successor relation, namely those defined through **substring-containment conditions**.

These come in two forms: one that bans the occurrence of a substring (negative literal), and one that requires the occurrence of at least one substring (positive literal), adopting the notation of Van Der Poel et al. (2024).

1. The former is defined as a conjunction of negative literals and corresponds to the classic Strictly Local (SL) class:

$$\bigwedge_{1 \leq i \leq n} \neg x_i$$

2. The latter is defined as a disjunction of positive literals and forms the complement of the SL class, namely coSL:

$$\bigvee_{1 \leq i \leq n} x_i$$

The next logical level is *Testable class*, here the relevant form being the Locally Testable (LT) class, constitutes a more permissive extension of the Boolean operators introduced above. Whereas the strict logical classes allow only formulas of the form

$$\bigwedge_{1\leq i\leq n} \neg x_i \quad \text{or} \quad \bigvee_{1\leq i\leq n} x_i \tag{1}$$

The LT class allows arbitrary Boolean combinations of such literals. That is, negative literals $\neg x_a$ may combine with positive literals x_b through conjunction, disjunction, implication, and related Boolean connectives, yielding formulas of greater logical complexity. For example:

$$\begin{aligned} &\neg x_a \wedge x_b, \qquad \neg x_a \vee x_b, \\ &\neg x_a \rightarrow x_b, \qquad (\neg x_a \wedge x_b) \rightarrow x_c \end{aligned} \tag{2}$$

While membership in these *testable* classes is determined simply by truth-valued Boolean conditions—that is, they are quantifier-free (Rogers and Lambert, 2019)—the next higher level of logical complexity introduces quantification over string candidates by counting occurrences up to a specified threshold, thereby forming the class of *Threshold Testable* languages. Formally, each factor x_i is no longer evaluated merely for presence or absence, but by whether its number of occurrences reaches a fixed threshold:

$$\#_w(x_i) < t \quad \text{or} \quad \#_w(x_i) \geq t \tag{3}$$

where $\#_w(x_i)$ denotes the number of occurrences of factor x_i in string w, and t is a fixed threshold. These languages remain star-free, since counting is sensitive only up to a fixed threshold and does not capture global modulo-n counting distinctions such as parity languages (Van Der Poel et al., 2024).

2.2 Piecewise Locally Testable (PLT)

The strict logical counterpart of PLT languages is SPL, which performs the same basic Boolean combinations: conjunctions of negative literals for SPL and disjunctions of positive literals for coSPL. Namely, expanding Equation 1, we obtain:

$$\begin{aligned} &\bigwedge_{1\leq i\leq m} \neg x_i, \qquad \bigvee_{1\leq i\leq m} x_i, \\ &\bigwedge_{1\leq j\leq n} \neg y_j, \qquad \bigvee_{1\leq j\leq n} y_j \end{aligned} \tag{4}$$

where x_i ranges over substrings and y_j ranges over subsequences; the negative conjunctive forms define forbidden patterns, while the positive disjunctive forms define licensing patterns.

SPL essentially combines adjacency with general precedence, allowing local and long-distance conditions to be stated simultaneously within a single constraint (Van Der Poel et al., 2024). As a result, the literals of the logical language may refer to sequences that are contiguous at some points and discontinuous at others. To capture this characterization in abstract form, Rogers and Lambert (2019) use the notation $\text{SPL}_{\text{j,k}}$, where j indicates the factor size of subsequences and k the factor size of substrings. This notation in turn reflects the model-theoretic representation given in Equation 5.

$$\mathcal{M}_{\Sigma}^{\lhd,\prec}(w) \stackrel{\text{def}}{=} \langle D^w, \lhd^w, \prec^w, \rtimes^w, \ltimes^w, P_\sigma^w / \sigma \in \Sigma \rangle \tag{5}$$

Figure 3: Model-theoretic representation of a string w over alphabet Σ, where D^w is the domain of positions, $\lhd^w$ is the successor relation, $\prec^w$ is the precedence relation, $\rtimes^w$ and $\ltimes^w$ are right and left boundary-markers, and P_σ^w is the unary predicate holding of positions labeled by symbol $\sigma \in \Sigma$.

In this linear model, factors are resolved into contiguous blocks of positions connected by $\lhd$, which are themselves ordered by $\prec$. Thus, precedence-defined factors may be viewed as ordered sequences of substring blocks. Accordingly, j also corresponds to the maximum number of substring blocks permitted within a factor.

Importantly, because this class encompasses both successor and precedence relations, fixing one factor size to 1 reduces the system to the corresponding strict propositional subclasses: $\text{SPL}_{\text{1,k}}$ yields the successor-based class SL_{k}, while $\text{SPL}_{\text{j,1}}$ yields the precedence-based class SP_{j}.

At the next level of logical complexity, namely the PLT classes, propositional logic is fully realized without the restrictions characteristic of the strict logical hierarchy. The limited Boolean combinations in Equation 4 are therefore generalized into richer propositional structures, extending Equation 2 to include both sequence-based atomic propositions (containment of subsequences y) and string-based atomic propositions (containment of substrings x). Formally, we obtain:

$$\begin{aligned} &\neg y_a \wedge x_b, \qquad (\neg x_a \wedge x_b) \rightarrow y_c, \\ &\neg x_a \wedge y_b, \qquad \neg y_a \vee y_b, \\ &\neg x_a \rightarrow y_b, \qquad (\neg y_a \wedge y_b) \rightarrow x_c \end{aligned} \tag{6}$$

Another important point to note about the SPL classes—the strict logical subclass of PLT—is their incomparability with TSL, the tier-based successor class at the same logical level. This is reflected in Figure 1 by the absence of connecting arrows between SPL and TSL, and likewise between PLT and TLT. Support for this distinction also comes from empirical evidence such as Latin liquid dissimilation, which TSL has been shown to capture whereas SPL fails to do so (Rogers and Lambert, 2019). This contrast will be revisited in the Discussion (Section 6) when evaluating whether the ordering-relation hierarchy between successor+precedence and either successor or precedence alone is weaker than distinctions of a different kind—namely, those between tier-based and non-tier-based languages.

2.3 Data - MLRegTest

Building on this formal characterization, MLRegTest (Van Der Poel et al., 2024) provides a generated benchmark of formal languages spanning the 16 classes outlined in Figure 1. The generated languages vary with respect to alphabet size (Σ), tier-projection alphabet size (τ), factor size (k), counting threshold (t), and ten distinct base patterns for each language class (i). Each language instance is encoded in the format:

$$\Sigma.\tau.\text{class}.k.t.i \tag{7}$$

In cases where tier projection was irrelevant, τ was set equal to the total alphabet size (i.e., $\Sigma = \tau$). Likewise, when counting was irrelevant, the threshold parameter was fixed at 1 (i.e., $t = 1$). Meanwhile, for the PLT and TPLT classes, the parameter t did not remain at 1, but varied in tandem with factor size k. Although not explicit in the original benchmark description, it was identified in the course of the present analysis that t corresponds to the subsequence factor size, while k corresponds to the substring factor size, mirroring the formal characterization of these classes encompassing both successor and precedence relations.

For the LTT and PLT languages, the values of the parameters in Equation 7 varied as summarized in Table 4. Of the 270 language varieties listed, we selected the 90 languages with alphabet size fixed at 64 ($\Sigma = 64$). This choice was motivated by the finding that alphabet size correlates positively with accuracy (Van Der Poel et al., 2024), and was therefore not expected to affect cross-architectural comparisons other than by raising the overall performance baseline. Each of these 90 languages was trained separately on three architectures (RNN[2], CNN, and LSTM), yielding a total of 270 trained models.

Class	Threshold t (Subseq. Factor)	Base pattern i (count)	Alphabet Σ	Substr. Factor k	Total
PLT	2	10	4,16,64	2	30
	2	10	4,16,64	4	30
	3	10	4,16,64	6	30
PLT total					**90**
LTT	2	10	4,16,64	2,4,6	90
	3	10	4,16,64	2,4	60
	5	10	4,16,64	2	30
LTT total					**180**

Table 4: Parameter configurations and language counts for PLT and LTT in MLRegTest.

For each language, **training sets** of three sizes (Large, Mid, and Small) were available, with each larger set properly containing the smaller ones. The present study uses only the Large split, since training-set size, like alphabet size, was reported to correlate positively with performance and was consequently not expected to materially affect the cross-architectural comparisons pursued here. Because training was formulated as a binary classification task rather than learning in the limit (Gold, 1967), each dataset contained both positive and negative strings in equal proportion.

	Split	Positive	Negative
Train	Large	50,000	50,000
Test	SR (short random)	50,000	50,000
	SA (short adversarial)	50,000	50,000
	LR (long random)	50,000	50,000
	LA (long adversarial)	50,000	50,000

Table 5: Summary statistics of the train and test sets used in the present study. These counts apply uniformly to each language variety considered here.

Meanwhile, **test sets** came in four varieties: Short Random (SR), Long Random (LR), Short Adversarial (SA), and Long Adversarial (LA). These differed along two dimensions, each operationalizing a criterion of successful learning (Li and Zhou, 2026): (i) length invariance and (ii) robustness to adversarial examples. In the Long test sets, string lengths were out of distribution relative to the training data (whose strings ranged from length 20 to 29), with LR and LA containing strings of length 31

[2]Hereafter, simple RNN is referred to simply as RNN.

to 50. By contrast, SR and SA used strings within the original training range of 20 to 29. In the adversarial sets, the edit distance between positive and negative strings was exactly 1, leaving minimal margin for classification errors. The random sets, by contrast, imposed no such constraint.

2.3.1 Base Pattern: PLT

The ten base patterns for each language class are worth examining, since they directly reveal how logical literals are composed. Following the generation code[3], PLT languages are built from seven primitive literals defined over the combined successor and precedence relations.

The generator draws symbols from a fixed alphabet of 64 characters consisting of lowercase letters, uppercase letters, and accented vowels[4]. In the present literals, however, only the symbols a, b, and c are used to compose the factors; the remaining symbols remain unconstrained and are thus freely insertable elsewhere in the string.

Under the relational structure in Equation 5, each inner list specifies a contiguous block of positions linked by the successor relation $\lhd$, while the outer list specifies an ordered sequence of such blocks linked by the precedence relation $\prec$. Hence, k denotes the size of each contiguous factor, and j denotes the number of precedence-ordered factors.The seven primitive literals are:

1. $l_1 = [[b], k * [c]] + (j * [[b], [c]])[: j - 2]$
2. $l_2 = [k * [a]] + (j - 1) * [[b]]$
3. $l_3 = [(k//2) * [a, b]] + (j - 1) * [[c]]$
4. $l_4 = [k * [b]]$
5. $l_5 = [[c]] + (j - 1) * [[a, a]]$
6. $l_6 = (j - 1) * [[a, b]] + [(k//2) * [b, a]]$
7. $l_7 = [[a, c]]$

The ten generated PLT patterns are then obtained by applying negation, union, implication, and the boundary-condition operator to these seven literals.

The **boundary-condition** operator functions as a wrapper around a primitive constraint, requiring that the string begin with the block $(k - 1) * [a]$ (i.e., a^{k-1}). In the present precedence-inclusive language class, this condition is instead implemented by forbidding the subsequence $[c] + (k - 1) * [a]$ (i.e., $c...a^{k-1}$).

Ultimately, the 10 language pattern in PLT classes are as follows:

0. $PLT_0 = \neg l_1$
1. $PLT_1 = \neg l_2$
2. $PLT_2 = l_3$
3. $PLT_3 = l_4 \vee l_5$
4. $PLT_4 = l_6 \rightarrow l_7$
5. $PLT_5 = \text{boundaryCondition}(\neg l_1)$
6. $PLT_6 = \text{boundaryCondition}(\neg l_2)$
7. $PLT_7 = \text{boundaryCondition}(l_3)$
8. $PLT_8 = \text{boundaryCondition}(l_4 \vee l_5)$
9. $PLT_9 = \text{boundaryCondition}(l_6 \rightarrow l_7)$

Essentially, PLT 5–9 are derived by wrapping a boundary condition around PLT 0–4. The same strategy is likewise used to derive LTT patterns 5–9, as discussed below.

2.3.2 Base Pattern: LTT

Because LTT evaluates the number of occurrences of local components up to a fixed threshold, an additional threshold parameter is built into the literals alongside a preferred or dispreferred structural configuration. The primitive literals of the LTT language class consist of four LTT-specific counting literals, given in 1–4, which follow the form in Equation 3, together with one propositional literal in 5.

1. $l_1 = \#_w(a^k) \geq t$
2. $l_2 = \#_w(\text{expr}) \geq t$
3. $l_3 = \#_w(a^k) = 0 \ \vee \ \#_w(a^k) \geq t$
4. $l_4 = \#_w(\text{expr}) = 0 \ \vee \ \#_w(\text{expr}) \geq t$
5. $l_5 = \bigvee_{i=0,2,\ldots,\text{base}-2} \left\langle (\sigma_i \sigma_{i+1})^{\lfloor k/2 \rfloor} \right\rangle$

The expr in 4 contains an additional parameter that is instantiated by another structural literal (Pattern 1 uses 5). The parameter base in 5 is an

[3] https://github.com/heinz-jeffrey/subregular-learning/blob/main/src/generate-subreglib.py

[4] The full alphabet is a,b,c,d,e,. . . ,x,y,z, A,B,C,D,E,. . . ,X,Y,Z, á,à,ǎ,é,è,ě,ó,ò,ǒ,ú,ù,ǔ.

iterating variable[5] whose purpose is to generate alternating adjacent symbol pairs repeated to length $k//2$. Each pair is formed from consecutive symbols in the underlying alphabet, namely $\sigma_i\sigma_{i+1}$; for example, (a, b) when $\texttt{base} = 2$, (a, b) and (c, d) when $\texttt{base} = 4$, and so forth.

The resulting substring candidates are then combined by disjunction, forming a single propositional expression. As base increases, additional alternating substrings built from the next adjacent symbol pairs are introduced, thereby increasing the number of positive literals in the disjunction.

As noted, the last five language patterns in LTT are obtained by applying the boundaryCondition wrapper, just as in PLT. The difference is that, since LTT is a locality-based language class, the boundary condition directly requires the string to begin with the block a^{k-1} rather than being implemented through the forbidding of subsequences. Consequently, we obtain the following ten patterns:

0. $LTT_0 = l_1$
1. $LTT_1 = \neg l_2(l_5)$
2. $LTT_2 = l_3$
3. $LTT_3 = l_3(l_5(\text{base} = 2))$
4. $LTT_4 = l_1 \wedge \neg l_2(l_5)$
5. $LTT_5 = \text{boundaryCondition}(l_1)$
6. $LTT_6 = \text{boundaryCondition}(\neg l_2(l_5))$
7. $LTT_7 = \text{boundaryCondition}(l_3)$
8. $LTT_8 = \text{boundaryCondition}\big(l_3\big(l_5(\text{base} = 2)\big)\big)$
9. $LTT_9 = \text{boundaryCondition}(l_1 \wedge \neg l_2(l_5))$

3 Previous Work on Neural Network Learnability of Subregular Languages

Recent years have seen renewed interest in evaluating the capacity of neural network (NN) models to learn formal languages. This line of work ranges from surveys of expressive capacity (Strobl et al., 2024), investigations of learnability for specific linguistic patterns (Nelson et al., 2020; Li and Zhou, 2025), studies on the learnability of mathematically defined formal language classes (Hua et al., 2025; Li and Zhou, 2026), and attempts to explicitly train models on subregular languages in order to induce useful inductive biases (Elsner and Black, 2026; Hu et al., 2025).

Nevertheless, much of the recent literature centers on large language models (LLMs) or, more broadly, transformer-based architectures (Elsner and Black, 2026; Strobl et al., 2024; Hua et al., 2025; Hu et al., 2025; Soulos et al., 2021). While such models are powerful, their internal learning dynamics are often less transparent, which can make them less suitable for fine-grained analyses of learnability. In line with this reasoning, studies concerned with interpretability and representation learning have often relied on simpler architectures such as RNNs (Li et al., 2024; Li and Zhou, 2026; Nelson et al., 2020). By contrast, transformer-based studies more commonly focus on improving capacity or architectural bias than on probing the mechanisms of learnability (Hu et al., 2025; Soulos et al., 2021; Elsner and Black, 2026).

A second limitation concerns architectural coverage. Even among studies using simpler models, most work concentrate on sequential models, leaving alternatives such as convolutional neural networks (CNNs) comparatively underexplored. The only two prior studies we identify as analyzing CNNs on formal-language tasks are Li and Zhou (2025) and Li and Zhou (2026). However, Li and Zhou (2025) focuses on tier-based ordering relations, examining the representational capacity of long-distance harmony patterns, where tier-based classes involve a distinct type of non-local dependency that is not directly comparable to the present subregular classes of interest (Heinz et al., 2011). Meanwhile, Li and Zhou (2026) evaluates CNNs on LTT, extending beyond the more common focus on simpler logical classes such as SP and SL (Avcu et al., 2017; Nelson et al., 2020; Hua et al., 2025), but does not consider PLT.

To our knowledge, Van Der Poel et al. (2024) is the first study to test the learnability of PLT using neural networks. However, as discussed in Section 1.2, its results remain limited by parameter imbalance across architectures and the absence of additional model types such as CNNs. The present study addresses these limitations under controlled settings while re-examining how learnability in neural networks is shaped by the interaction between logical complexity and ordering relations—particularly when these two dimensions stand in tension with one another.

Prior studies have largely examined individual

[5] For example, in Pattern 4, base is fixed to the hyperparameter value 2.

dimensions of the hierarchy, for example by comparing SL with SP (Avcu et al., 2017) with respect to ordering relations. Cross-dimensional contrasts have also appeared in comparisons between SL and LTT (Li and Zhou, 2026), though not as their central focus. Moreover, SL and LTT do not instantiate the same reciprocal structure found between LTT and PLT, where PLT properly contains LTT while the two differ in logical complexity and ordering relations in contrasting ways. Cases of this kind have not been systematically investigated, and examining this intersection constitutes the principal novelty of the present study.

4 Experimental Setup

Expanding upon the parameter-count table in Table 3, for the RNN and LSTM models we retained the settings that do not affect parameter count and that yielded the best performance in the grid search of Van Der Poel et al. (2024), namely the learning rate, optimizer, number of training epochs, loss function, and dropout rate.

For the newly introduced CNN model, the learning rate was set to the lower of the two values considered in the original grid search (0.01 and 0.001) as a conservative choice to encourage stable optimization for the newly introduced 22×3 Conv1D architecture. The Adam optimizer was selected from among Adam, RMSProp, and SGD as a widely used adaptive optimizer that provides a strong default baseline across neural architectures.

The number of training epochs and the loss function were aligned with those used for the RNN and LSTM models in order to maintain comparability across architectures. The dropout rate was set to 0.1, chosen from the original alternatives of 0.0 and 0.1, a conservative choice to reduce the risk of overfitting. The full configuration is reported in Table 6.

Testing over the four aforementioned test sets (SR, LR, SA, and LA) followed the original prediction and evaluation code, which reported results over eleven confusion-matrix-based metrics: TN (true negatives), TP (true positives), FP (false positives), FN (false negatives), TPR (true positive rate[6]), FPR (false positive rate[7]), Accuracy[8], Precision[9], F-score[10], AUC (area under the ROC curve), and the Brier score. As the analysis in Van Der Poel et al. (2024) primarily focuses on the Accuracy metric, the present study follows the same convention; nevertheless, the full evaluation results are to be provided in Appendix A.

The only modification concerned the prediction stage: the original non-batched implementation resulted in a cpu memory exhaustion, so inference was performed in batches of size 64, matching the training setup.

[6] $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$
[7] $\mathrm{FP}/(\mathrm{FP}+\mathrm{TN})$
[8] $(\mathrm{TP}+\mathrm{TN})/(\mathrm{TP}+\mathrm{TN}+\mathrm{FP}+\mathrm{FN})$
[9] $\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$
[10] $\mathrm{TP}/(\mathrm{TP}+0.5(\mathrm{FP}+\mathrm{FN}))$

	Network Type		
Hyperparameters	**CNN**	**Simple RNN**	**LSTM**
Embedding Dimension	32	32	32
Number of FF Layers	–	4	2
Architecture	22×3 Conv1D	–	–
Parameter Count	18,424	19,020	16,930
Learning rate	0.0001	0.0001	0.0001
Optimizer	Adam	Adam	RMSprop
Training Epochs	64	64	64
Loss Function	BCE	BCE	BCE
Drop out	0.1	0.1	0.0

Table 6: Model and Training Configuration

5 Results

Given that our central question is whether ordering relations can outweigh, or at least rival, logical complexity depending on architecture, the analysis is conducted at two levels.

First, at the cross-language level, we compare PLT and LTT to determine whether performance differs between the two language classes. Second, at the within-language level, we examine whether performance varies as a function of factor size in both PLT and LTT, and additionally threshold size in LTT. These internal parameters also serve to test the sensitivity of learnability outcomes to ordering relations, which may be inferred through preferences for, or indifference to, locality constraints, beyond the cross-language comparison alone.

5.1 Across-Language Comparison

The language varieties investigated in the present study follow the naming format given in Equation 8 (see also Equation 7 and Section 2.3).

$$64.64.\mathrm{class}.k.t.i \tag{8}$$

Accordingly, four principal independent variables are available for pairwise analysis of the training results against the eleven evaluation metrics introduced in Section 4.

In the main text, Accuracy is treated as the principal evaluation metric and serves as the basis for the inferential analyses reported below. The remaining ten metrics are reported in Appendix A as supplementary descriptive summary tables, organized into confusion-matrix counts (TP, TN, FP, FN), derived rate-based measures (TPR, FPR, Precision, and F-score), and probabilistic evaluation metrics (AUC and Brier score).

At the cross-language level, we compare performance across language classes (PLT vs. LTT) by network type, while controlling for factor size k and threshold size t through averaging across matched values of these parameters. We further examine whether Accuracy differs by language pattern (Sections 2.3.1 and 2.3.2) in interaction with network type. Lastly, performance is analyzed across the four test conditions SR, LR, SA, and LA, providing comparisons of network type by test condition.

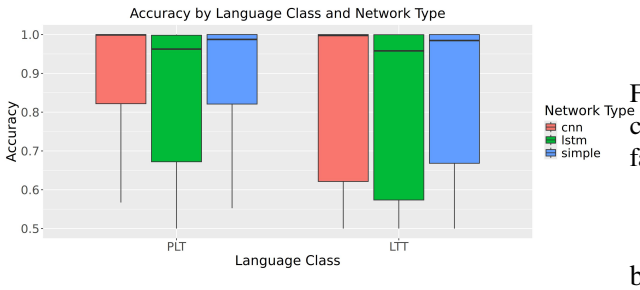

Figure 4: Mean accuracy by language class (LTT vs. PLT) and network architecture, averaged across test type, pattern, factor size (k), and threshold size (t).

We first examine whether different neural architectures treat the learning of language classes differently; that is, whether the relative learnability of the two languages remains invariant across architectures. Figure 4 provides partial support for Hypothesis 2, which predicted that the relative learnability ranking between LTT and PLT would vary across architectures. However, the results reveal an unexpected pattern: reducing the sequential processing advantage appears to make ordering relations less, rather than more, salient, thereby reinforcing the precedence of logical complexity in determining learnability.

The trend is interpretable to some extent. The most locality-biased architecture, CNN, exhibits the largest performance gap between PLT and LTT, followed by the simple RNN, while the gap is smallest for the LSTM. In other words, sensitivity to the distinction between the two language classes decreases progressively from CNN to RNN to LSTM, in line with the assumed architectural gradient of inductive bias from local to sequential processing.

Nevertheless, the overall ranking of learnability remains stable across all architectures, with PLT consistently outperforming LTT. More strikingly, diminishing sequential processing capacity does not amplify the importance of ordering relations. Instead, the inferiority of LTT relative to PLT becomes greater as the architecture becomes more locality-oriented. This is somewhat ironic, given that LTT is defined solely over the successor relation, whereas PLT is defined over both precedence and successor relations.

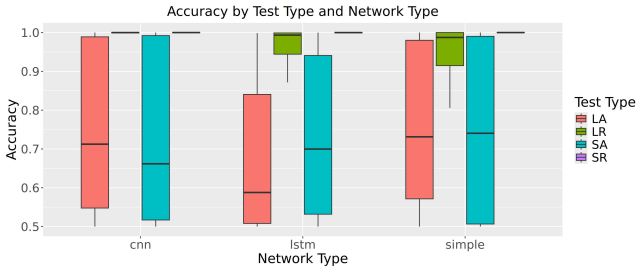

Figure 5: Mean accuracy by test type and network architecture, averaged across language classes, patterns, factor size (k), and threshold size (t).

The performance differences across test types by architecture in Figure 5 once again confirm that the CNN is the most locality-sensitive architecture. The performance gap between the in-distribution conditions (SR, SA) and the out-of-distribution conditions is largest for the CNN, followed by the simple RNN, and smallest for the LSTM.

Thus, cross-architectural differences appear to systematically encode sensitivity to ordering relations, insofar as preference for locally structured test conditions aligns with each model's position on the locality spectrum of inductive bias. This makes the earlier finding—namely, that the PLT–LTT gap is reinforced rather than attenuated across architectures—particularly intriguing.

The results suggest that improvements in architectures designed for longer-term dependency tracking do not substantially enhance performance on non-local relations. Rather, the more robust effect seems to be that locality-biased architectures perform especially well when the task itself is local. In this sense, architectural changes toward greater sequential capacity may matter less than the extent to which local biases can be exploited by the target dependency structure.

This interpretation also echoes earlier findings: the superior performance of CNNs on SL languages reported by Li and Zhou (2026), and the unexpectedly weak performance of LSTMs on SP languages observed by Avcu et al. (2017).

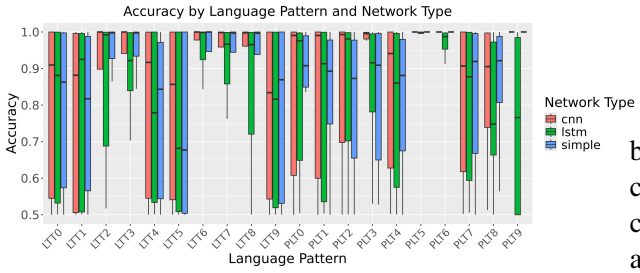

Figure 6: Mean accuracy by language pattern and network architecture, averaged across test type, factor size (k), and threshold size (t).

Meanwhile, the apparently skewed distributions in Figure 4—where the median often lies near ceiling level while the error range extends substantially downward—invite closer examination within each language class. Figure 6 breaks down accuracy by individual patterns that constitute each language.

Within LTT, Patterns 2, 3, 6, 7, and 8 achieve consistently higher accuracy than Patterns 0, 1, 4, 5, and 9. At the same time, for these higher-performing patterns, LSTM falls behind to a noticeably greater extent than either the CNN or simple RNN, which remain comparatively close to one another. In other words, the weaker overall performance of the LSTM in Figure 4, as well as its reduced sensitivity to between-language differences, appears to be driven largely by these five otherwise more learnable LTT pattern types.

Among these, Patterns 6, 7, and 8 share the property that they are the `boundaryCondition`-wrapped counterparts of the more primitive Patterns 1, 2, and 3, respectively. For the latter two pairings, the trend is unsurprising, since Patterns 2 and 3 were likewise identified as comparatively easy to learn. Why these properties are particularly difficult for the LSTM remains unclear.

More noteworthy is the substantially lower accuracy of Pattern 1 relative to Pattern 6, despite the general expectation that adding a boundary condition should increase formal complexity and thereby reduce learnability. Recall that Pattern 1 is an embedded instance of a negative literal, which can be written explicitly as:

$$LTT_1 = \neg\#_w \left(\bigvee_{i=0,2,\ldots,\text{base}-2} \langle (\sigma_i \sigma_{i+1})^{\lfloor k/2 \rfloor} \rangle \right) \geq t$$

This contrast may suggest that the effect of boundary condition, when combined with negation, can partially mitigate rather than amplify structural complexity. That is, the boundary-conditioned variant may become easier to learn than its more primitive negative counterpart.

A similar tendency can be observed in PLT. Patterns 5 and 6, which exhibit near-ceiling performance across all architectures in Figure 6, are likewise the wrapped counterparts of the negated primitive literals 2 and 5. By contrast, patterns composed solely of these literals, namely PLT 0 and PLT 1, show comparatively lower performance.

Further evidence comes from Pattern 9, which—like the five LTT patterns identified above—shows the largest performance gap between the LSTM and the CNN/RNN group. This pattern is the boundary condition-ed counterpart of Pattern 4, whose performance is comparatively uniform across architectures.

Taken together, the pattern-wise analysis suggests two tentative generalizations. First, `boundaryCondition` appears to increase the performance gap between the LSTM and more locality-biased architectures. Second, when combined with negation, the wrapper may in some cases increase overall learnability rather than decrease it.

5.2 Within-Language Comparison

Within-language analyses fix the class variable in Equation 8 and focus on the remaining structural parameters not already examined in the cross-language comparison.

We investigate how performance varies with factor size k in both PLT and LTT, and additionally with threshold size t in LTT, across network types.

5.2.1 Effect of Factor Size

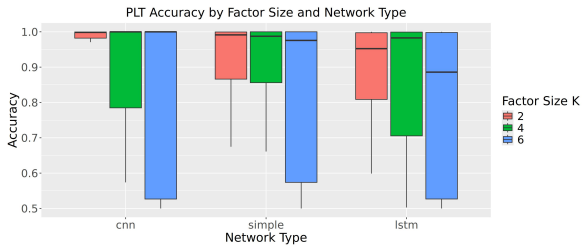

Figure 7: Effect of factor size (k) on mean accuracy in PLT across network architectures.

The results in Figure 7 show that factor size has the strongest effect on the CNN, the architecture with the strongest local inductive bias. CNN performance declines most steeply as factor size increases, followed by the less locally oriented simple RNN, while the LSTM exhibits the smallest decrease. This pattern mirrors the cross-language comparison: sensitivity to local learning patterns increases with the strength of an architecture's local inductive bias.

Moreover, the cross-architectural comparison of performance variation with increasing window size is more consistent with the view that CNNs benefit from locally structured language patterns than with the alternative expectation that LSTMs should gain a corresponding advantage on structures requiring longer dependencies. This interpretation also echoes the conclusion drawn earlier.

Figure 8 shows that the same ordering of steepness across architectures is preserved for LTT languages.

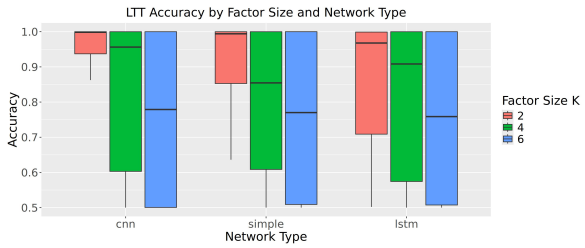

Figure 8: Effect of factor size (k) on mean accuracy in LTT across network architectures.

5.2.2 Effect of Threshold Size in LTT

The effect of threshold size across architectures broadly follows a similar pattern. The CNN, which has the strongest local inductive bias, is also the most sensitive to locality-related variation, showing the greatest performance differences across values

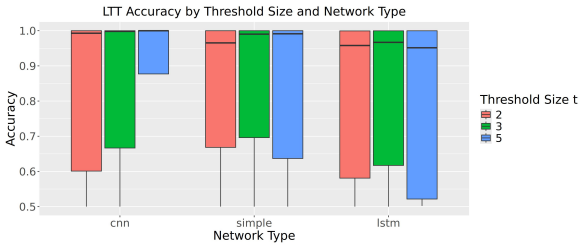

Figure 9: Effect of threshold size (t) on mean accuracy in LTT across network architectures.

of t. This contrasts with the smaller variation observed in the simple RNN and especially the LSTM, the latter exhibiting the greatest invariance.

The direction of this effect in the CNN, however, is unexpected: accuracy decreases as threshold size becomes smaller. By contrast, the simple RNN and LSTM display the more intuitive trend, with smaller threshold values being easier to learn. This peculiarity merits further investigation, as it is not yet fully explained within the scope of the present study.

6 Discussion

The general conclusion is that logical complexity consistently prevails whenever the ordering-relation hierarchy and the logical-complexity hierarchy conflict or are reversely aligned. The stronger an architecture's local inductive bias, the more transparently this dominance is revealed. Thus, our hypothesis appears to have been only partially correct: it was correct insofar as sequentially oriented architectures do obscure the impact of ordering relations.

However, the direction of this effect was opposite from our initial expectation. Rather than increasing the performance of architectures oriented toward longer-distance dependencies and thereafter producing invariance between PLT and LTT, the observed invariance instead appears to arise from a mitigation of the effect of logical complexity. Consequently, once the sequential advantage diminishes, the dominance of logical complexity becomes more apparent, and the learnability gap widens accordingly.

This finding ultimately raises the question of what constitutes the ordering-relation hierarchy, or set-relational power, between PLT and LTT. More fundamentally, it raises the question of what properties characterize language classes that make use of both precedence and successor relations ($\prec, +1$), as opposed to those relying on only one of these

relations. This also invites comparison with other inclusion-based hierarchies along different ordering relation, such as the distinction between tier-based and non-tier-based classes.

Recall that in the average accuracy results of the original experiment reported in Van Der Poel et al. (2024) and summarized earlier in Table 1, the relationship between accuracy and set inclusion for tier-based versus non-tier-based classes followed the expected reverse ordering (e.g., SL $\subseteq$ TSL, with accuracy 0.862 for SL and 0.837 for TSL), whereas for PT $\subseteq$ PLT, the accuracy ordering aligned with the set-theoretic direction (PLT: 0.865; PT: 0.816).

The fact that the expected learnability relation PT $\subseteq$ PLT did not pattern like other set-theoretic relations suggests that restricting generalization within SPL languages—whether in terms of subsequence factors (j) or substring factors (k) in $\mathrm{SPL}_{j,k}$—may not exert the same kind of influence as more substantive hierarchical distinctions, such as those between tier-based and non-tier-based languages. This is particularly intriguing given that TSL and PLT are incomparable classes with different empirical coverage, as illustrated by long-distance phonotactic dissimilation patterns such as Latin liquid dissimilation (Rogers and Lambert, 2019).

References

Enes Avcu, Chihiro Shibata, and Jeffrey Heinz. 2017. Subregular complexity and deep learning. *arXiv preprint arXiv:1705.05940*.

Micha Elsner and Daniel S. Black. 2026. Sub-regular inductive biases in a phonological transformer. In *Proceedings of the Annual Meetings on Phonology*, volume 2. University of Massachusetts Amherst Libraries.

E. Mark Gold. 1967. Language identification in the limit. *Information and Control*, 10(5):447–474.

Jeffrey Heinz, Chetan Rawal, and Herbert G. Tanner. 2011. Tier-based strictly local constraints for phonology. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 58–64.

Michael Y Hu, Jackson Petty, Chuan Shi, William Merrill, and Tal Linzen. 2025. Between circuits and chomsky: Pre-pretraining on formal languages imparts linguistic biases. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9691–9709.

Wenyue Hua, Tyler Wong, Fei Sun, Liangming Pan, Adam Jardine, and William Yang Wang. 2025. Inductionbench: Llms fail in the simplest complexity class. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 26526–26546.

Jane Li and Alan Zhou. 2026. (r) nns too expressive? no relationship between neural architectures' theoretical expressivity and their empirical performance. *Linguistics Vanguard*, (0).

Jane Li and Alan Tiantian Zhou. 2025. Cnns that robustly compute vowel harmony do not explicitly represent phonological tiers. In *Proceedings of the Society for Computation in Linguistics 2025*, pages 392–394.

Jiahui Li, Kyle Rawlins, and Paul Smolensky. 2024. What representations do rnns learn and use from morpho-phonological processes?: An exploration of pca and pc neutralizations on turkish vowel harmony. *Society for Computation in Linguistics*, 7(1).

Max Nelson, Hossep Dolatian, Jonathan Rawski, and Brandon Prickett. 2020. Probing rnn encoder-decoder generalization of subregular functions using reduplication. In *Proceedings of the Society for Computation in Linguistics 2020*, pages 167–178.

James Rogers and Dakotah Lambert. 2019. Some classes of sets of structures definable without quantifiers. In *Proceedings of the 16th Meeting on the Mathematics of Language*, pages 63–77.

Paul Soulos, Sudha Rao, Colin Smith, Eric Rosen, Asli Celikyilmaz, R. Thomas McCoy, others, and Paul Smolensky. 2021. Structural biases for improving transformers on translation into morphologically rich languages. In *Proceedings of the 4th Workshop on Technologies for MT of Low Resource Languages (LoResMT2021)*, pages 52–67.

Lena Strobl, William Merrill, Gail Weiss, David Chiang, and Dana Angluin. 2024. What formal languages can transformers express? a survey. *Transactions of the Association for Computational Linguistics*, 12:543–561.

Sam Van Der Poel, Dakotah Lambert, Kalina Kostyszyn, Tiantian Gao, Rahul Verma, Derek Andersen, Joanne Chau, Emily Peterson, Cody St Clair, Paul Fodor, and 1 others. 2024. Mlregtest: A benchmark for the machine learning of regular languages. *Journal of Machine Learning Research*, 25(283):1–45.

A Appendix

Architecture	Class	Mean	Median	SD	Q1	Q3
GRU	LTT	0.899	0.998	0.169	0.871	1.000
	PLT	0.936	0.999	0.135	0.963	1.000
LSTM	LTT	0.830	0.945	0.194	0.630	0.999
	PLT	0.829	0.941	0.191	0.643	0.997
Simple RNN	LTT	0.815	0.890	0.187	0.626	0.995
	PLT	0.846	0.927	0.171	0.695	0.994
Transformer	LTT	0.824	0.895	0.180	0.650	0.994
	PLT	0.853	0.921	0.161	0.718	0.996

Table 7: Summary statistics corresponding to the box-plot comparison of LTT and PLT accuracies in Figure 2

A.1 Confusion-Matrix Metrics

Table 8: Statistics for TP by language class and network architecture

Architecture	Class	Mean	Median	SD	Q1	Q3
CNN	LTT	45935.2	49998	12417.1	49972	50000
	PLT	45188.2	50000	13553.1	49944	50000
LSTM	LTT	44885.6	49999	13034.3	49192	50000
	PLT	41791.7	49988	16332.0	45179	50000
Simple RNN	LTT	46033.0	49999	9960.4	49080	50000
	PLT	45489.1	49996	10770.5	48031	50000

Table 9: Statistics for FP by language class and network architecture

Architecture	Class	Mean	Median	SD	Q1	Q3
CNN	LTT	10696.2	14	18527.5	1	10835
	PLT	5415.3	5	13438.2	0	200
LSTM	LTT	12184.4	80	19022.7	0	21616
	PLT	7547.0	187	14596.7	3	5662
Simple RNN	LTT	11646.3	31	17972.3	1	21016
	PLT	6929.0	24	12466.3	0	9262

Table 10: Statistics for TN by language class and network architecture

Architecture	Class	Mean	Median	SD	Q1	Q3
CNN	LTT	39303.9	49986	18527.5	39165	49999
	PLT	44584.7	49995	13438.2	49800	50000
LSTM	LTT	37815.6	49920	19022.7	28384	50000
	PLT	42453.0	49814	14596.7	44338	49997
Simple RNN	LTT	38353.7	49969	17972.3	28984	49999
	PLT	43071.0	49977	12466.3	40738	50000

Table 11: Statistics for FN by language class and network architecture

Architecture	Class	Mean	Median	SD	Q1	Q3
CNN	LTT	4064.9	2	12417.1	0	28
	PLT	4811.8	0	13553.1	0	56
LSTM	LTT	5114.4	1	13034.3	0	809
	PLT	8208.3	12	16332.0	0	4821
Simple RNN	LTT	3967.0	1	9960.4	0	920
	PLT	4510.9	4	10770.5	0	1969

Table 12: Statistics for TP by test type and network architecture

Architecture	Test Type	Mean	Median	SD	Q1	Q3
CNN	LA	41738.0	49990	16543.8	48447	50000
	LR	49900.5	49997	353.5	49982	50000
	SA	41120.0	49999	17686.7	49833	50000
	SR	49986.2	50000	58.2	49993	50000
LSTM	LA	38091.1	49932	18507.3	27963	50000
	LR	46829.0	49989	8351.9	47443	50000
	SA	41313.8	49998	17307.9	47528	50000
	SR	49183.4	50000	5644.3	49986	50000
Simple RNN	LA	42941.2	48504	10165.8	37660	50000
	LR	47664.9	49990	4723.1	48606	50000
	SA	42806.9	50000	16047.5	49826	50000
	SR	49993.7	50000	18.4	49997	50000

Table 13: Statistics for FP by test type and network architecture

Architecture	Test Type	Mean	Median	SD	Q1	Q3
CNN	LA	17219.5	2124	20408.8	1	38853
	LR	127.4	9	527.5	0	19
	SA	18367.9	1021	21295.2	0	44082
	SR	28.6	3	71.1	0	12
LSTM	LA	20950.9	13510	21116.4	106	46658
	LR	1881.3	45	5459.1	1	474
	SA	19200.0	8250	20413.9	3	43422
	SR	522.2	4	4563.0	0	39
Simple RNN	LA	18227.4	16147	16770.5	1045	29720
	LR	3359.1	16	7588.7	1	2427
	SA	18150.9	1196	21416.0	0	43964
	SR	558.2	1	5229.3	0	9

Table 14: Statistics for TN by test type and network architecture

Architecture	Test Type	Mean	Median	SD	Q1	Q3
CNN	LA	32780.5	47876	20408.8	11147	49999
	LR	49872.6	49992	527.5	49982	50000
	SA	31632.1	48980	21295.2	5918	50000
	SR	49971.4	49997	71.1	49989	50000
LSTM	LA	29049.1	36491	21116.4	3342	49894
	LR	48118.7	49955	5459.1	49527	49999
	SA	30799.0	41751	20413.9	6579	49997
	SR	49477.8	49996	4563.0	49962	50000
Simple RNN	LA	31772.6	33854	16770.5	20280	48955
	LR	46640.9	49984	7588.7	47573	49999
	SA	31849.1	48805	21416.0	6036	50000
	SR	49441.8	49999	5229.3	49992	50000

Table 15: Statistics for FN by test type and network architecture

Architecture	Test Type	Mean	Median	SD	Q1	Q3
CNN	LA	8262.0	10	16543.8	0	1553
	LR	99.5	4	353.5	0	18
	SA	8880.0	1	17686.7	0	167
	SR	13.8	0	58.2	0	7
LSTM	LA	11908.9	68	18507.3	0	22037
	LR	3171.0	11	8351.9	0	2557
	SA	8686.2	2	17307.9	0	2472
	SR	816.6	1	5644.3	0	14
Simple RNN	LA	7058.8	1496	10165.8	0	12340
	LR	2335.1	11	4723.1	0	1394
	SA	7193.1	0	16047.5	0	174
	SR	6.3	0	18.4	0	3

Table 16: Statistics for TP by language pattern and network architecture

Architecture	Pattern	Mean	Median	SD	Q1	Q3
CNN	LTT0	49821.2	49990	466.2	49969	50000
CNN	LTT1	28481.2	41543	23131.1	1064	49987
CNN	LTT2	39435.3	49993	18732.0	39823	49999
CNN	LTT3	42043.4	49993	14893.6	44124	50000
CNN	LTT4	49699.2	49993	850.8	49950	50000
CNN	LTT5	49918.7	49992	177.8	49947	49999
CNN	LTT6	49999.9	50000	0.3	50000	50000
CNN	LTT7	49999.7	50000	0.8	50000	50000
CNN	LTT8	49998.9	50000	2.2	49999	50000
CNN	LTT9	49954.0	49994	92.2	49982	50000
CNN	PLT0	34594.2	49047	22087.4	10784	49987
CNN	PLT1	34579.7	49056	22099.7	9974	49995
CNN	PLT2	49999.3	50000	1.5	50000	50000
CNN	PLT3	49861.0	49976	210.3	49807	49989
CNN	PLT4	32976.8	44126	20709.4	12778	49985
CNN	PLT5	50000.0	50000	0.0	50000	50000
CNN	PLT6	50000.0	50000	0.0	50000	50000
CNN	PLT7	50000.0	50000	0.0	50000	50000
CNN	PLT8	49871.4	49973	237.9	49926	50000
CNN	PLT9	50000.0	50000	0.0	50000	50000
LSTM	LTT0	49497.9	49998	959.7	49639	50000
LSTM	LTT1	28437.6	42822	22864.6	852	49920
LSTM	LTT2	38039.0	49226	18222.0	31745	49968
LSTM	LTT3	40633.0	46199	13989.7	39692	49808
LSTM	LTT4	48727.5	50000	3061.1	50000	50000
LSTM	LTT5	49999.6	50000	1.4	50000	50000
LSTM	LTT6	49595.6	50000	1369.9	50000	50000
LSTM	LTT7	48084.5	49999	6413.5	49983	50000
LSTM	LTT8	45978.6	50000	13620.5	49999	50000
LSTM	LTT9	49863.1	50000	384.2	49989	50000
LSTM	PLT0	34406.4	47600	20488.1	14890	49937
LSTM	PLT1	30389.1	41326	22433.4	3572	49966
LSTM	PLT2	49972.2	49999	40.9	49931	50000
LSTM	PLT3	48637.4	49956	3081.0	49898	49988
LSTM	PLT4	30234.1	38200	21739.9	7493	49946
LSTM	PLT5	50000.0	50000	0.0	50000	50000
LSTM	PLT6	49927.0	50000	170.5	50000	50000
LSTM	PLT7	49993.3	50000	13.9	49998	50000
LSTM	PLT8	48507.0	49983	2328.2	46871	49996
LSTM	PLT9	25850.2	27456	21407.8	0	50000
Simple RNN	LTT0	46438.0	49856	4867.7	42725	50000
Simple RNN	LTT1	33382.3	47329	19759.8	17291	49987
Simple RNN	LTT2	43961.8	49970	11953.2	43612	49996
Simple RNN	LTT3	44000.6	49956	12936.0	47751	49992
Simple RNN	LTT4	44059.2	48116	6649.3	36338	50000
Simple RNN	LTT5	48828.2	50000	3269.3	49771	50000
Simple RNN	LTT6	50000.0	50000	0.0	50000	50000
Simple RNN	LTT7	49998.0	50000	5.3	50000	50000
Simple RNN	LTT8	49999.2	50000	1.8	50000	50000
Simple RNN	LTT9	49662.2	50000	703.7	49847	50000
Simple RNN	PLT0	39460.2	48863	17067.8	36067	49965
Simple RNN	PLT1	37703.1	46471	17020.4	27266	49996
Simple RNN	PLT2	47986.4	48532	2173.6	45534	50000
Simple RNN	PLT3	45493.8	49927	6589.2	36820	49985
Simple RNN	PLT4	36305.8	45192	17956.0	25256	49958
Simple RNN	PLT5	49999.9	50000	0.3	50000	50000
Simple RNN	PLT6	50000.0	50000	0.0	50000	50000
Simple RNN	PLT7	49433.1	49891	797.4	48847	50000
Simple RNN	PLT8	48508.7	49298	2157.8	48352	49996
Simple RNN	PLT9	50000.0	50000	0.0	50000	50000

Table 17: Statistics for FP by language pattern and network architecture

Architecture	Pattern	Mean	Median	SD	Q1	Q3
CNN	LTT0	21005.0	7394	22740.6	8	45502
CNN	LTT1	834.5	368	1266.5	8	736
CNN	LTT2	1.9	0	5.0	0	0
CNN	LTT3	0.0	0	0.0	0	0
CNN	LTT4	20932.3	5341	22903.1	11	45482
CNN	LTT5	22176.2	13732	23174.4	8	45867
CNN	LTT6	5242.3	6	12720.2	0	2204
CNN	LTT7	7268.4	165	14027.6	8	4125
CNN	LTT8	7067.5	239	13758.8	12	3884
CNN	LTT9	22433.5	16385	23238.6	9	45696
CNN	PLT0	2.9	2	3.4	0	6
CNN	PLT1	0.4	0	0.8	0	0
CNN	PLT2	13746.2	694	20468.7	9	30243
CNN	PLT3	7125.8	216	15472.5	88	1838
CNN	PLT4	2.6	0	5.4	0	1
CNN	PLT5	36.8	0	98.1	0	1
CNN	PLT6	145.8	0	348.5	0	5
CNN	PLT7	17886.8	9323	20841.4	13	38208
CNN	PLT8	15202.2	9305	17831.3	189	26118
CNN	PLT9	3.2	0	8.3	0	0
LSTM	LTT0	20470.0	9889	22187.4	14	45521
LSTM	LTT1	99.3	0	205.3	0	29
LSTM	LTT2	2221.3	0	7509.9	0	4
LSTM	LTT3	1871.2	0	4749.1	0	3
LSTM	LTT4	22340.1	22112	22152.1	18	46055
LSTM	LTT5	24673.2	31825	23570.9	15	49137
LSTM	LTT6	8413.7	105	15445.8	2	6533
LSTM	LTT7	7270.8	1179	13029.6	61	7230
LSTM	LTT8	10928.3	1910	16887.4	67	14387
LSTM	LTT9	23556.2	17550	23738.7	12	48046
LSTM	PLT0	66.7	3	138.8	2	27
LSTM	PLT1	7.0	0	14.8	0	3
LSTM	PLT2	15194.3	1917	19912.9	182	29734
LSTM	PLT3	13379.9	6231	17006.4	520	17788
LSTM	PLT4	2452.7	1	5731.3	0	3
LSTM	PLT5	743.2	212	1328.5	14	408
LSTM	PLT6	2723.1	1163	3384.5	207	4373
LSTM	PLT7	20474.8	12241	21723.4	61	40614
LSTM	PLT8	19909.7	25195	19224.0	222	29793
LSTM	PLT9	518.6	46	732.8	0	1430
Simple RNN	LTT0	17713.5	7636	20164.3	9	37316
Simple RNN	LTT1	5711.5	243	7475.5	0	10267
Simple RNN	LTT2	1890.5	0	4210.5	0	193
Simple RNN	LTT3	1808.6	5	4464.6	0	593
Simple RNN	LTT4	17569.3	5856	20336.2	11	34926
Simple RNN	LTT5	25403.9	29964	23055.7	4	49624
Simple RNN	LTT6	8211.6	7	15703.2	0	5352
Simple RNN	LTT7	7853.8	281	15010.3	6	5563
Simple RNN	LTT8	8448.8	323	15505.6	13	6159
Simple RNN	LTT9	21851.9	11901	23293.6	1	46894
Simple RNN	PLT0	3312.8	682	4409.0	0	8549
Simple RNN	PLT1	4156.7	1445	5303.8	0	8554
Simple RNN	PLT2	16945.8	10424	18501.4	4	32316
Simple RNN	PLT3	13240.1	9007	14888.5	330	21376
Simple RNN	PLT4	4090.1	1243	5856.2	0	6363
Simple RNN	PLT5	40.5	1	97.0	0	19
Simple RNN	PLT6	145.8	0	348.5	0	5
Simple RNN	PLT7	16233.9	7254	19539.8	9	32314
Simple RNN	PLT8	11120.9	5108	14185.1	56	17663
Simple RNN	PLT9	3.6	0	9.1	0	1

Table 18: Statistics for TN by language pattern and network architecture

Architecture	Pattern	Mean	Median	SD	Q1	Q3
CNN	LTT0	28995.0	42606	22740.6	4498	49992
CNN	LTT1	49165.5	49633	1266.5	49264	49992
CNN	LTT2	49998.1	50000	5.0	50000	50000
CNN	LTT3	50000.0	50000	0.0	50000	50000
CNN	LTT4	29067.7	44659	22903.1	4518	49989
CNN	LTT5	27823.8	36269	23174.4	4133	49992
CNN	LTT6	44757.8	49995	12720.2	47796	50000
CNN	LTT7	42731.6	49836	14027.6	45875	49992
CNN	LTT8	42932.5	49761	13758.8	46116	49988
CNN	LTT9	27566.5	33616	23238.6	4305	49991
CNN	PLT0	49997.1	49999	3.4	49994	50000
CNN	PLT1	49999.6	50000	0.8	50000	50000
CNN	PLT2	36253.8	49307	20468.7	19757	49991
CNN	PLT3	42874.2	49784	15472.5	48162	49913
CNN	PLT4	49997.4	50000	5.4	49999	50000
CNN	PLT5	49963.3	50000	98.1	50000	50000
CNN	PLT6	49854.2	50000	348.5	49995	50000
CNN	PLT7	32113.2	40677	20841.4	11792	49988
CNN	PLT8	34797.8	40696	17831.3	23882	49812
CNN	PLT9	49996.8	50000	8.3	50000	50000
LSTM	LTT0	29530.0	40111	22187.4	4479	49986
LSTM	LTT1	49900.7	50000	205.3	49971	50000
LSTM	LTT2	47778.7	50000	7509.9	49996	50000
LSTM	LTT3	48128.8	50000	4749.1	49997	50000
LSTM	LTT4	27659.9	27888	22152.1	3946	49983
LSTM	LTT5	25326.8	18176	23570.9	863	49986
LSTM	LTT6	41586.3	49895	15445.8	43467	49998
LSTM	LTT7	42729.3	48821	13029.6	42770	49939
LSTM	LTT8	39071.7	48090	16887.4	35613	49933
LSTM	LTT9	26443.8	32450	23738.7	1955	49989
LSTM	PLT0	49933.3	49997	138.8	49973	49999
LSTM	PLT1	49993.0	50000	14.8	49997	50000
LSTM	PLT2	34805.8	48084	19912.9	20266	49818
LSTM	PLT3	36620.1	43769	17006.4	32213	49480
LSTM	PLT4	47547.3	49999	5731.3	49997	50000
LSTM	PLT5	49256.8	49788	1328.5	49593	49986
LSTM	PLT6	47276.9	48837	3384.5	45627	49793
LSTM	PLT7	29525.3	37759	21723.4	9386	49940
LSTM	PLT8	30090.3	24805	19224.0	20207	49778
LSTM	PLT9	49481.4	49955	732.8	48570	50000
Simple RNN	LTT0	32286.5	42365	20164.3	12684	49991
Simple RNN	LTT1	44288.5	49758	7475.5	39734	50000
Simple RNN	LTT2	48109.5	50000	4210.5	49807	50000
Simple RNN	LTT3	48191.4	49995	4464.6	49408	50000
Simple RNN	LTT4	32430.7	44144	20336.2	15074	49989
Simple RNN	LTT5	24596.1	20037	23055.7	376	49996
Simple RNN	LTT6	41788.4	49994	15703.2	44648	50000
Simple RNN	LTT7	42146.2	49720	15010.3	44438	49994
Simple RNN	LTT8	41551.2	49678	15505.6	43841	49987
Simple RNN	LTT9	28148.1	38099	23293.6	3106	49999
Simple RNN	PLT0	46687.3	49319	4409.0	41451	50000
Simple RNN	PLT1	45843.3	48555	5303.8	41446	50000
Simple RNN	PLT2	33054.3	39576	18501.4	17684	49996
Simple RNN	PLT3	36759.9	40993	14888.5	28625	49671
Simple RNN	PLT4	45909.9	48757	5856.2	43637	50000
Simple RNN	PLT5	49959.5	49999	97.0	49981	50000
Simple RNN	PLT6	49854.2	50000	348.5	49995	50000
Simple RNN	PLT7	33766.1	42746	19539.8	17686	49991
Simple RNN	PLT8	38879.1	44893	14185.1	32337	49944
Simple RNN	PLT9	49996.4	50000	9.1	49999	50000

Table 19: Statistics for FN by language pattern and network architecture

Architecture	Pattern	Mean	Median	SD	Q1	Q3
CNN	LTT0	178.8	10	466.2	0	31
CNN	LTT1	21518.8	8458	23131.1	13	48936
CNN	LTT2	10564.7	8	18732.0	1	10177
CNN	LTT3	7956.6	7	14893.6	0	5877
CNN	LTT4	300.8	8	850.8	0	50
CNN	LTT5	81.3	8	177.8	1	53
CNN	LTT6	0.1	0	0.3	0	0
CNN	LTT7	0.3	0	0.8	0	0
CNN	LTT8	1.1	0	2.2	0	1
CNN	LTT9	46.0	6	92.2	0	18
CNN	PLT0	15405.8	953	22087.4	13	39216
CNN	PLT1	15420.3	944	22099.7	5	40026
CNN	PLT2	0.8	0	1.5	0	1
CNN	PLT3	139.0	24	210.3	11	193
CNN	PLT4	17023.3	5875	20709.4	15	37222
CNN	PLT5	0.0	0	0.0	0	0
CNN	PLT6	0.0	0	0.0	0	0
CNN	PLT7	0.0	0	0.0	0	0
CNN	PLT8	128.6	28	237.9	0	74
CNN	PLT9	0.0	0	0.0	0	0
LSTM	LTT0	502.1	2	959.7	0	362
LSTM	LTT1	21562.4	7178	22864.6	80	49148
LSTM	LTT2	11961.0	775	18222.0	32	18255
LSTM	LTT3	9367.0	3801	13989.7	192	10308
LSTM	LTT4	1272.5	0	3061.1	0	0
LSTM	LTT5	0.4	0	1.4	0	0
LSTM	LTT6	404.4	0	1369.9	0	0
LSTM	LTT7	1915.5	1	6413.5	0	17
LSTM	LTT8	4021.4	0	13620.5	0	1
LSTM	LTT9	136.9	0	384.2	0	12
LSTM	PLT0	15593.6	2401	20488.1	63	35111
LSTM	PLT1	19610.9	8674	22433.4	35	46428
LSTM	PLT2	27.8	1	40.9	0	70
LSTM	PLT3	1362.6	45	3081.0	12	102
LSTM	PLT4	19765.9	11800	21739.9	54	42507
LSTM	PLT5	0.0	0	0.0	0	0
LSTM	PLT6	73.0	0	170.5	0	0
LSTM	PLT7	6.7	0	13.9	0	3
LSTM	PLT8	1493.0	17	2328.2	0	3129
LSTM	PLT9	24149.8	22544	21407.8	0	50000
Simple RNN	LTT0	3562.0	144	4867.7	0	7276
Simple RNN	LTT1	16617.7	2672	19759.8	13	32709
Simple RNN	LTT2	6038.2	31	11953.2	4	6388
Simple RNN	LTT3	5999.4	44	12936.0	8	2249
Simple RNN	LTT4	5940.8	1884	6649.3	0	13662
Simple RNN	LTT5	1171.8	0	3269.3	0	230
Simple RNN	LTT6	0.0	0	0.0	0	0
Simple RNN	LTT7	2.0	0	5.3	0	0
Simple RNN	LTT8	0.8	0	1.8	0	0
Simple RNN	LTT9	337.8	1	703.7	0	153
Simple RNN	PLT0	10539.8	1138	17067.8	35	13933
Simple RNN	PLT1	12296.9	3529	17020.4	4	22735
Simple RNN	PLT2	2013.6	1469	2173.6	0	4466
Simple RNN	PLT3	4506.2	73	6589.2	15	13181
Simple RNN	PLT4	13694.2	4809	17956.0	43	24744
Simple RNN	PLT5	0.1	0	0.3	0	0
Simple RNN	PLT6	0.0	0	0.0	0	0
Simple RNN	PLT7	566.9	110	797.4	0	1154
Simple RNN	PLT8	1491.3	703	2157.8	4	1649
Simple RNN	PLT9	0.0	0	0.0	0	0

A.2 Derived Rate-Based Measures

Table 20: Statistics for TPR by language class and network architecture

Architecture	Class	Mean	Median	SD	Q1	Q3
CNN	LTT	0.919	1.000	0.248	0.999	1.000
	PLT	0.904	1.000	0.271	0.999	1.000
LSTM	LTT	0.898	1.000	0.261	0.984	1.000
	PLT	0.836	1.000	0.327	0.904	1.000
Simple RNN	LTT	0.921	1.000	0.199	0.982	1.000
	PLT	0.910	1.000	0.215	0.961	1.000

Table 21: Statistics for FPR by language class and network architecture

Architecture	Class	Mean	Median	SD	Q1	Q3
CNN	LTT	0.214	0.000	0.371	0.000	0.217
	PLT	0.108	0.000	0.269	0.000	0.004
LSTM	LTT	0.244	0.002	0.380	0.000	0.432
	PLT	0.151	0.004	0.292	0.000	0.113
Simple RNN	LTT	0.233	0.001	0.359	0.000	0.420
	PLT	0.139	0.000	0.249	0.000	0.185

Table 22: Statistics for Precision by language class and network architecture

Architecture	Class	Mean	Median	SD	Q1	Q3
CNN	LTT	0.872	1.000	0.197	0.703	1.000
	PLT	0.934	1.000	0.149	0.995	1.000
LSTM	LTT	0.862	0.998	0.200	0.698	1.000
	PLT	0.902	0.995	0.163	0.893	1.000
Simple RNN	LTT	0.858	0.999	0.199	0.681	1.000
	PLT	0.898	1.000	0.160	0.832	1.000

Table 23: Statistics for F-Score by language class and network architecture

Architecture	Class	Mean	Median	SD	Q1	Q3
CNN	LTT	0.853	0.998	0.242	0.692	1.000
	PLT	0.875	0.999	0.257	0.846	1.000
LSTM	LTT	0.826	0.957	0.250	0.687	1.000
	PLT	0.800	0.962	0.304	0.721	0.998
Simple RNN	LTT	0.858	0.985	0.202	0.712	1.000
	PLT	0.879	0.987	0.207	0.820	1.000

Table 24: Statistics for TPR by test type and network architecture

Architecture	Test Type	Mean	Median	SD	Q1	Q3
CNN	LA	0.835	1.000	0.331	0.969	1.000
	LR	0.998	1.000	0.007	1.000	1.000
	SA	0.822	1.000	0.354	0.997	1.000
	SR	1.000	1.000	0.001	1.000	1.000
LSTM	LA	0.762	0.999	0.370	0.559	1.000
	LR	0.937	1.000	0.167	0.949	1.000
	SA	0.826	1.000	0.346	0.951	1.000
	SR	0.984	1.000	0.113	1.000	1.000
Simple RNN	LA	0.859	0.970	0.203	0.753	1.000
	LR	0.953	1.000	0.094	0.972	1.000
	SA	0.856	1.000	0.321	0.997	1.000
	SR	1.000	1.000	0.000	1.000	1.000

Table 25: Statistics for FPR by test type and network architecture

Architecture	Test Type	Mean	Median	SD	Q1	Q3
CNN	LA	0.344	0.042	0.408	0.000	0.777
	LR	0.003	0.000	0.011	0.000	0.000
	SA	0.367	0.020	0.426	0.000	0.882
	SR	0.001	0.000	0.001	0.000	0.000
LSTM	LA	0.419	0.270	0.422	0.002	0.933
	LR	0.038	0.001	0.109	0.000	0.009
	SA	0.384	0.165	0.408	0.000	0.868
	SR	0.010	0.000	0.091	0.000	0.001
Simple RNN	LA	0.365	0.323	0.335	0.021	0.594
	LR	0.067	0.000	0.152	0.000	0.049
	SA	0.363	0.024	0.428	0.000	0.879
	SR	0.011	0.000	0.105	0.000	0.000

Table 26: Statistics for Precision by test type and network architecture

Architecture	Test Type	Mean	Median	SD	Q1	Q3
CNN	LA	0.783	0.824	0.209	0.536	1.000
	LR	0.998	1.000	0.010	1.000	1.000
	SA	0.787	0.927	0.221	0.530	1.000
	SR	0.999	1.000	0.001	1.000	1.000
LSTM	LA	0.756	0.766	0.212	0.515	0.979
	LR	0.971	0.999	0.077	0.990	1.000
	SA	0.781	0.852	0.211	0.535	1.000
	SR	0.994	1.000	0.049	0.999	1.000
Simple RNN	LA	0.743	0.704	0.191	0.565	0.977
	LR	0.947	1.000	0.105	0.954	1.000
	SA	0.799	0.976	0.221	0.531	1.000
	SR	0.994	1.000	0.052	1.000	1.000

Table 27: Statistics for F-Score by test type and network architecture

Architecture	Test Type	Mean	Median	SD	Q1	Q3
CNN	LA	0.733	0.753	0.279	0.667	0.989
	LR	0.998	1.000	0.006	0.999	1.000
	SA	0.710	0.714	0.300	0.667	0.992
	SR	1.000	1.000	0.001	1.000	1.000
LSTM	LA	0.644	0.679	0.294	0.667	0.837
	LR	0.939	0.993	0.153	0.944	0.999
	SA	0.706	0.731	0.292	0.670	0.942
	SR	0.982	1.000	0.112	0.999	1.000
Simple RNN	LA	0.775	0.768	0.170	0.667	0.980
	LR	0.945	0.987	0.084	0.918	1.000
	SA	0.742	0.794	0.288	0.667	0.990
	SR	0.996	1.000	0.035	1.000	1.000

Table 28: Statistics for TPR by language pattern and network architecture

Architecture	Pattern	Mean	Median	SD	Q1	Q3
CNN	LTT0	0.996	1.000	0.009	0.999	1.000
CNN	LTT1	0.570	0.831	0.463	0.021	1.000
CNN	LTT2	0.789	1.000	0.375	0.796	1.000
CNN	LTT3	0.841	1.000	0.298	0.882	1.000
CNN	LTT4	0.994	1.000	0.017	0.999	1.000
CNN	LTT5	0.998	1.000	0.004	0.999	1.000
CNN	LTT6	1.000	1.000	0.000	1.000	1.000
CNN	LTT7	1.000	1.000	0.000	1.000	1.000
CNN	LTT8	1.000	1.000	0.000	1.000	1.000
CNN	LTT9	0.999	1.000	0.002	1.000	1.000
CNN	PLT0	0.692	0.981	0.442	0.216	1.000
CNN	PLT1	0.692	0.981	0.442	0.199	1.000
CNN	PLT2	1.000	1.000	0.000	1.000	1.000
CNN	PLT3	0.997	1.000	0.004	0.996	1.000
CNN	PLT4	0.660	0.883	0.414	0.256	1.000
CNN	PLT5	1.000	1.000	0.000	1.000	1.000
CNN	PLT6	1.000	1.000	0.000	1.000	1.000
CNN	PLT7	1.000	1.000	0.000	1.000	1.000
CNN	PLT8	0.997	0.999	0.005	0.999	1.000
CNN	PLT9	1.000	1.000	0.000	1.000	1.000
LSTM	LTT0	0.990	1.000	0.019	0.993	1.000
LSTM	LTT1	0.569	0.856	0.457	0.017	0.998
LSTM	LTT2	0.761	0.985	0.364	0.635	0.999
LSTM	LTT3	0.813	0.924	0.280	0.794	0.996
LSTM	LTT4	0.975	1.000	0.061	1.000	1.000
LSTM	LTT5	1.000	1.000	0.000	1.000	1.000
LSTM	LTT6	0.992	1.000	0.027	1.000	1.000
LSTM	LTT7	0.962	1.000	0.128	1.000	1.000
LSTM	LTT8	0.920	1.000	0.272	1.000	1.000
LSTM	LTT9	0.997	1.000	0.008	1.000	1.000
LSTM	PLT0	0.688	0.952	0.410	0.298	0.999
LSTM	PLT1	0.608	0.827	0.449	0.071	0.999
LSTM	PLT2	0.999	1.000	0.001	0.999	1.000
LSTM	PLT3	0.973	0.999	0.062	0.998	1.000
LSTM	PLT4	0.605	0.764	0.435	0.150	0.999
LSTM	PLT5	1.000	1.000	0.000	1.000	1.000
LSTM	PLT6	0.999	1.000	0.003	1.000	1.000
LSTM	PLT7	1.000	1.000	0.000	1.000	1.000
LSTM	PLT8	0.970	1.000	0.047	0.937	1.000
LSTM	PLT9	0.517	0.549	0.428	0.000	1.000
Simple RNN	LTT0	0.929	0.997	0.097	0.854	1.000
Simple RNN	LTT1	0.668	0.947	0.395	0.346	1.000
Simple RNN	LTT2	0.879	0.999	0.239	0.872	1.000
Simple RNN	LTT3	0.880	0.999	0.259	0.955	1.000
Simple RNN	LTT4	0.881	0.962	0.133	0.727	1.000
Simple RNN	LTT5	0.977	1.000	0.065	0.995	1.000
Simple RNN	LTT6	1.000	1.000	0.000	1.000	1.000
Simple RNN	LTT7	1.000	1.000	0.000	1.000	1.000
Simple RNN	LTT8	1.000	1.000	0.000	1.000	1.000
Simple RNN	LTT9	0.993	1.000	0.014	0.997	1.000
Simple RNN	PLT0	0.789	0.977	0.341	0.721	0.999
Simple RNN	PLT1	0.754	0.929	0.340	0.545	1.000
Simple RNN	PLT2	0.960	0.971	0.043	0.911	1.000
Simple RNN	PLT3	0.910	0.999	0.132	0.736	1.000
Simple RNN	PLT4	0.726	0.904	0.359	0.505	0.999
Simple RNN	PLT5	1.000	1.000	0.000	1.000	1.000
Simple RNN	PLT6	1.000	1.000	0.000	1.000	1.000
Simple RNN	PLT7	0.989	0.998	0.016	0.977	1.000
Simple RNN	PLT8	0.970	0.986	0.043	0.967	1.000
Simple RNN	PLT9	1.000	1.000	0.000	1.000	1.000

Table 29: Statistics for FPR by language pattern and network architecture

Architecture	Pattern	Mean	Median	SD	Q1	Q3
CNN	LTT0	0.420	0.148	0.455	0.000	0.910
CNN	LTT1	0.017	0.007	0.025	0.000	0.015
CNN	LTT2	0.000	0.000	0.000	0.000	0.000
CNN	LTT3	0.000	0.000	0.000	0.000	0.000
CNN	LTT4	0.419	0.107	0.458	0.000	0.910
CNN	LTT5	0.444	0.275	0.463	0.000	0.917
CNN	LTT6	0.105	0.000	0.254	0.000	0.044
CNN	LTT7	0.145	0.003	0.281	0.000	0.083
CNN	LTT8	0.141	0.005	0.275	0.000	0.078
CNN	LTT9	0.449	0.328	0.465	0.000	0.914
CNN	PLT0	0.000	0.000	0.000	0.000	0.000
CNN	PLT1	0.000	0.000	0.000	0.000	0.000
CNN	PLT2	0.275	0.014	0.409	0.000	0.605
CNN	PLT3	0.143	0.004	0.309	0.002	0.037
CNN	PLT4	0.000	0.000	0.000	0.000	0.000
CNN	PLT5	0.001	0.000	0.002	0.000	0.000
CNN	PLT6	0.003	0.000	0.007	0.000	0.000
CNN	PLT7	0.358	0.186	0.417	0.000	0.764
CNN	PLT8	0.304	0.186	0.357	0.004	0.522
CNN	PLT9	0.000	0.000	0.000	0.000	0.000
LSTM	LTT0	0.409	0.198	0.444	0.000	0.910
LSTM	LTT1	0.002	0.000	0.004	0.000	0.001
LSTM	LTT2	0.044	0.000	0.150	0.000	0.000
LSTM	LTT3	0.037	0.000	0.095	0.000	0.000
LSTM	LTT4	0.447	0.442	0.443	0.000	0.921
LSTM	LTT5	0.493	0.636	0.471	0.000	0.983
LSTM	LTT6	0.168	0.002	0.309	0.000	0.131
LSTM	LTT7	0.145	0.024	0.261	0.001	0.145
LSTM	LTT8	0.219	0.038	0.338	0.001	0.288
LSTM	LTT9	0.471	0.351	0.475	0.000	0.961
LSTM	PLT0	0.001	0.000	0.003	0.000	0.001
LSTM	PLT1	0.000	0.000	0.000	0.000	0.000
LSTM	PLT2	0.304	0.038	0.398	0.004	0.595
LSTM	PLT3	0.268	0.125	0.340	0.010	0.356
LSTM	PLT4	0.049	0.000	0.115	0.000	0.000
LSTM	PLT5	0.015	0.004	0.027	0.000	0.008
LSTM	PLT6	0.054	0.023	0.068	0.004	0.087
LSTM	PLT7	0.409	0.245	0.434	0.001	0.812
LSTM	PLT8	0.398	0.504	0.384	0.004	0.596
LSTM	PLT9	0.010	0.001	0.015	0.000	0.029
Simple RNN	LTT0	0.354	0.153	0.403	0.000	0.746
Simple RNN	LTT1	0.114	0.005	0.150	0.000	0.205
Simple RNN	LTT2	0.038	0.000	0.084	0.000	0.004
Simple RNN	LTT3	0.036	0.000	0.089	0.000	0.012
Simple RNN	LTT4	0.351	0.117	0.407	0.000	0.699
Simple RNN	LTT5	0.508	0.599	0.461	0.000	0.992
Simple RNN	LTT6	0.164	0.000	0.314	0.000	0.107
Simple RNN	LTT7	0.157	0.006	0.300	0.000	0.111
Simple RNN	LTT8	0.169	0.006	0.310	0.000	0.123
Simple RNN	LTT9	0.437	0.238	0.466	0.000	0.938
Simple RNN	PLT0	0.066	0.014	0.088	0.000	0.171
Simple RNN	PLT1	0.083	0.029	0.106	0.000	0.171
Simple RNN	PLT2	0.339	0.208	0.370	0.000	0.646
Simple RNN	PLT3	0.265	0.180	0.298	0.007	0.428
Simple RNN	PLT4	0.082	0.025	0.117	0.000	0.127
Simple RNN	PLT5	0.001	0.000	0.002	0.000	0.000
Simple RNN	PLT6	0.003	0.000	0.007	0.000	0.000
Simple RNN	PLT7	0.325	0.145	0.391	0.000	0.646
Simple RNN	PLT8	0.222	0.102	0.284	0.001	0.353
Simple RNN	PLT9	0.000	0.000	0.000	0.000	0.000

Table 30: Statistics for Precision by language pattern and network architecture

Architecture	Pattern	Mean	Median	SD	Q1	Q3
CNN	LTT0	0.776	0.882	0.235	0.524	1.000
CNN	LTT1	0.908	0.964	0.126	0.891	0.995
CNN	LTT2	0.978	1.000	0.104	1.000	1.000
CNN	LTT3	1.000	1.000	0.000	1.000	1.000
CNN	LTT4	0.777	0.906	0.236	0.524	1.000
CNN	LTT5	0.766	0.821	0.240	0.521	1.000
CNN	LTT6	0.935	1.000	0.138	0.962	1.000
CNN	LTT7	0.910	0.997	0.159	0.928	1.000
CNN	LTT8	0.912	0.995	0.156	0.934	1.000
CNN	LTT9	0.764	0.801	0.241	0.523	1.000
CNN	PLT0	0.981	1.000	0.067	1.000	1.000
CNN	PLT1	1.000	1.000	0.000	1.000	1.000
CNN	PLT2	0.847	0.986	0.216	0.623	1.000
CNN	PLT3	0.917	0.996	0.170	0.964	0.998
CNN	PLT4	1.000	1.000	0.000	1.000	1.000
CNN	PLT5	0.999	1.000	0.002	1.000	1.000
CNN	PLT6	0.997	1.000	0.007	1.000	1.000
CNN	PLT7	0.798	0.863	0.221	0.567	1.000
CNN	PLT8	0.815	0.862	0.199	0.657	0.996
CNN	PLT9	1.000	1.000	0.000	1.000	1.000
LSTM	LTT0	0.778	0.855	0.232	0.516	1.000
LSTM	LTT1	0.992	1.000	0.022	0.991	1.000
LSTM	LTT2	0.968	1.000	0.108	1.000	1.000
LSTM	LTT3	0.968	1.000	0.079	1.000	1.000
LSTM	LTT4	0.755	0.697	0.233	0.517	1.000
LSTM	LTT5	0.742	0.613	0.241	0.504	1.000
LSTM	LTT6	0.899	0.998	0.171	0.891	1.000
LSTM	LTT7	0.903	0.969	0.142	0.875	0.999
LSTM	LTT8	0.866	0.943	0.179	0.779	0.999
LSTM	LTT9	0.754	0.754	0.241	0.510	1.000
LSTM	PLT0	0.993	1.000	0.023	0.999	1.000
LSTM	PLT1	0.997	1.000	0.010	1.000	1.000
LSTM	PLT2	0.825	0.963	0.211	0.627	0.996
LSTM	PLT3	0.829	0.889	0.180	0.742	0.990
LSTM	PLT4	0.951	1.000	0.118	1.000	1.000
LSTM	PLT5	0.986	0.996	0.025	0.992	1.000
LSTM	PLT6	0.952	0.977	0.058	0.920	0.996
LSTM	PLT7	0.772	0.812	0.225	0.553	0.999
LSTM	PLT8	0.763	0.665	0.212	0.607	0.995
LSTM	PLT9	0.979	0.984	0.023	0.965	0.999
Simple RNN	LTT0	0.790	0.882	0.223	0.540	1.000
Simple RNN	LTT1	0.881	0.995	0.158	0.778	1.000
Simple RNN	LTT2	0.956	1.000	0.100	0.996	1.000
Simple RNN	LTT3	0.962	1.000	0.090	0.989	1.000
Simple RNN	LTT4	0.785	0.888	0.228	0.534	1.000
Simple RNN	LTT5	0.729	0.609	0.238	0.502	1.000
Simple RNN	LTT6	0.904	1.000	0.175	0.924	1.000
Simple RNN	LTT7	0.905	0.994	0.166	0.916	1.000
Simple RNN	LTT8	0.899	0.994	0.172	0.906	1.000
Simple RNN	LTT9	0.769	0.839	0.240	0.516	1.000
Simple RNN	PLT0	0.913	0.987	0.147	0.852	1.000
Simple RNN	PLT1	0.901	0.945	0.121	0.826	1.000
Simple RNN	PLT2	0.791	0.824	0.210	0.594	1.000
Simple RNN	PLT3	0.804	0.850	0.196	0.630	0.993
Simple RNN	PLT4	0.915	0.976	0.120	0.879	1.000
Simple RNN	PLT5	0.999	1.000	0.002	1.000	1.000
Simple RNN	PLT6	0.997	1.000	0.007	1.000	1.000
Simple RNN	PLT7	0.810	0.886	0.212	0.605	1.000
Simple RNN	PLT8	0.850	0.914	0.173	0.733	0.999
Simple RNN	PLT9	1.000	1.000	0.000	1.000	1.000

Table 31: Statistics for F-score by language pattern and network architecture

Architecture	Pattern	Mean	Median	SD	Q1	Q3
CNN	LTT0	0.852	0.919	0.154	0.687	1.000
CNN	LTT1	0.592	0.868	0.447	0.041	0.996
CNN	LTT2	0.814	1.000	0.340	0.841	1.000
CNN	LTT3	0.875	1.000	0.252	0.925	1.000
CNN	LTT4	0.852	0.921	0.154	0.687	1.000
CNN	LTT5	0.846	0.886	0.158	0.685	1.000
CNN	LTT6	0.960	1.000	0.089	0.980	1.000
CNN	LTT7	0.945	0.998	0.101	0.962	1.000
CNN	LTT8	0.946	0.998	0.099	0.964	1.000
CNN	LTT9	0.845	0.874	0.159	0.686	1.000
CNN	PLT0	0.718	0.990	0.420	0.354	1.000
CNN	PLT1	0.718	0.990	0.420	0.333	1.000
CNN	PLT2	0.902	0.993	0.141	0.768	1.000
CNN	PLT3	0.947	0.996	0.110	0.981	0.999
CNN	PLT4	0.707	0.933	0.387	0.403	1.000
CNN	PLT5	1.000	1.000	0.001	1.000	1.000
CNN	PLT6	0.999	1.000	0.003	1.000	1.000
CNN	PLT7	0.871	0.921	0.144	0.724	1.000
CNN	PLT8	0.885	0.919	0.127	0.793	0.997
CNN	PLT9	1.000	1.000	0.000	1.000	1.000
LSTM	LTT0	0.852	0.897	0.151	0.681	1.000
LSTM	LTT1	0.598	0.919	0.453	0.033	0.996
LSTM	LTT2	0.782	0.992	0.331	0.672	0.999
LSTM	LTT3	0.842	0.919	0.257	0.837	0.998
LSTM	LTT4	0.830	0.820	0.152	0.679	1.000
LSTM	LTT5	0.831	0.760	0.159	0.671	1.000
LSTM	LTT6	0.933	0.999	0.108	0.928	1.000
LSTM	LTT7	0.918	0.968	0.112	0.876	0.999
LSTM	LTT8	0.841	0.966	0.264	0.785	0.999
LSTM	LTT9	0.837	0.849	0.159	0.675	1.000
LSTM	PLT0	0.731	0.975	0.380	0.452	0.997
LSTM	PLT1	0.643	0.903	0.444	0.132	0.999
LSTM	PLT2	0.890	0.981	0.137	0.770	0.998
LSTM	PLT3	0.883	0.915	0.117	0.802	0.995
LSTM	PLT4	0.628	0.858	0.423	0.251	0.996
LSTM	PLT5	0.993	0.998	0.013	0.996	1.000
LSTM	PLT6	0.974	0.988	0.031	0.955	0.997
LSTM	PLT7	0.854	0.894	0.148	0.712	0.999
LSTM	PLT8	0.838	0.799	0.133	0.733	0.972
LSTM	PLT9	0.565	0.700	0.436	0.000	0.985
Simple RNN	LTT0	0.834	0.861	0.143	0.701	0.998
Simple RNN	LTT1	0.673	0.845	0.358	0.444	0.988
Simple RNN	LTT2	0.894	0.997	0.226	0.926	1.000
Simple RNN	LTT3	0.892	0.997	0.232	0.930	1.000
Simple RNN	LTT4	0.808	0.821	0.149	0.667	0.971
Simple RNN	LTT5	0.812	0.754	0.154	0.668	1.000
Simple RNN	LTT6	0.940	1.000	0.112	0.956	1.000
Simple RNN	LTT7	0.941	0.997	0.106	0.952	1.000
Simple RNN	LTT8	0.937	0.997	0.110	0.947	1.000
Simple RNN	LTT9	0.846	0.892	0.158	0.679	1.000
Simple RNN	PLT0	0.800	0.915	0.319	0.834	0.990
Simple RNN	PLT1	0.770	0.903	0.302	0.680	0.979
Simple RNN	PLT2	0.854	0.884	0.136	0.731	0.977
Simple RNN	PLT3	0.847	0.917	0.161	0.690	0.996
Simple RNN	PLT4	0.744	0.887	0.325	0.593	0.980
Simple RNN	PLT5	1.000	1.000	0.001	1.000	1.000
Simple RNN	PLT6	0.999	1.000	0.003	1.000	1.000
Simple RNN	PLT7	0.876	0.928	0.137	0.748	0.996
Simple RNN	PLT8	0.897	0.924	0.109	0.834	0.987
Simple RNN	PLT9	1.000	1.000	0.000	1.000	1.000

A.3 Probabilistic Evaluation Metrics

Table 32: Statistics for AUC by language class and network architecture

Architecture	Class	Mean	Median	SD	Q1	Q3
CNN	LTT	0.861	0.999	0.199	0.655	1.000
	PLT	0.906	0.999	0.169	0.881	1.000
LSTM	LTT	0.842	0.976	0.198	0.606	1.000
	PLT	0.865	0.981	0.184	0.718	0.999
Simple RNN	LTT	0.854	0.990	0.192	0.721	1.000
	PLT	0.901	0.994	0.155	0.868	1.000

Table 33: Statistics for Brier Score by language class and network architecture

Architecture	Class	Mean	Median	SD	Q1	Q3
CNN	LTT	0.145	0.002	0.202	0.000	0.373
	PLT	0.101	0.001	0.175	0.000	0.165
LSTM	LTT	0.166	0.040	0.200	0.000	0.405
	PLT	0.143	0.036	0.179	0.002	0.250
Simple RNN	LTT	0.149	0.013	0.189	0.000	0.285
	PLT	0.109	0.011	0.159	0.000	0.162

Table 34: Statistics for AUC by test type and network architecture

Architecture	Test Type	Mean	Median	SD	Q1	Q3
CNN	LA	0.762	0.765	0.202	0.550	0.989
	LR	1.000	1.000	0.001	0.999	1.000
	SA	0.743	0.702	0.207	0.541	0.992
	SR	1.000	1.000	0.000	1.000	1.000
LSTM	LA	0.699	0.630	0.185	0.526	0.883
	LR	0.968	0.998	0.083	0.987	0.999
	SA	0.740	0.729	0.194	0.548	0.955
	SR	0.991	1.000	0.058	1.000	1.000
Simple RNN	LA	0.769	0.772	0.181	0.592	0.983
	LR	0.954	0.995	0.084	0.935	1.000
	SA	0.762	0.790	0.213	0.508	0.993
	SR	0.994	1.000	0.052	1.000	1.000

Table 35: Statistics for Brier Score by test type and network architecture

Architecture	Test Type	Mean	Median	SD	Q1	Q3
CNN	LA	0.250	0.276	0.203	0.011	0.451
	LR	0.002	0.000	0.005	0.000	0.001
	SA	0.269	0.326	0.208	0.008	0.478
	SR	0.000	0.000	0.001	0.000	0.000
LSTM	LA	0.316	0.380	0.180	0.155	0.483
	LR	0.040	0.006	0.070	0.001	0.046
	SA	0.270	0.279	0.193	0.049	0.462
	SR	0.008	0.000	0.038	0.000	0.001
Simple RNN	LA	0.241	0.248	0.178	0.019	0.407
	LR	0.051	0.011	0.079	0.000	0.078
	SA	0.247	0.234	0.211	0.009	0.492
	SR	0.003	0.000	0.026	0.000	0.000

Table 36: Statistics for AUC by pattern type and network architecture

Architecture	Pattern	Mean	Median	SD	Q1	Q3
CNN	LTT0	0.799	0.953	0.225	0.545	1.000
	LTT1	0.803	0.953	0.225	0.512	0.999
	LTT2	0.894	1.000	0.187	0.898	1.000
	LTT3	0.929	1.000	0.137	0.958	1.000
	LTT4	0.803	0.966	0.225	0.545	1.000
	LTT5	0.786	0.888	0.227	0.542	1.000
	LTT6	0.948	1.000	0.127	0.978	1.000
	LTT7	0.932	1.000	0.137	0.966	1.000
	LTT8	0.934	0.998	0.134	0.967	1.000
	LTT9	0.785	0.877	0.227	0.547	1.000
	PLT0	0.848	0.990	0.218	0.620	1.000
	PLT1	0.846	0.991	0.221	0.600	1.000
	PLT2	0.868	0.993	0.199	0.732	1.000
	PLT3	0.936	0.998	0.141	0.985	0.999
	PLT4	0.844	0.968	0.202	0.641	1.000
	PLT5	1.000	1.000	0.001	1.000	1.000
	PLT6	0.999	1.000	0.003	1.000	1.000
	PLT7	0.841	0.936	0.197	0.682	1.000
	PLT8	0.874	0.921	0.157	0.823	0.999
	PLT9	1.000	1.000	0.000	1.000	1.000
LSTM	LTT0	0.802	0.923	0.218	0.552	1.000
	LTT1	0.801	0.955	0.221	0.560	0.999
	LTT2	0.866	0.993	0.185	0.698	1.000
	LTT3	0.907	0.958	0.137	0.885	0.998
	LTT4	0.787	0.850	0.216	0.555	1.000
	LTT5	0.770	0.804	0.231	0.527	1.000
	LTT6	0.914	0.999	0.153	0.935	1.000
	LTT7	0.927	0.985	0.130	0.927	0.999
	LTT8	0.870	0.965	0.171	0.771	0.999
	LTT9	0.776	0.880	0.234	0.527	1.000
	PLT0	0.867	0.991	0.196	0.704	0.999
	PLT1	0.813	0.943	0.225	0.547	0.999
	PLT2	0.858	0.991	0.199	0.707	0.998
	PLT3	0.911	0.962	0.127	0.902	0.999
	PLT4	0.830	0.903	0.196	0.629	0.999
	PLT5	0.993	0.998	0.013	0.996	1.000
	PLT6	0.973	0.988	0.033	0.959	0.998
	PLT7	0.806	0.900	0.214	0.605	0.999
	PLT8	0.805	0.789	0.188	0.691	0.996
	PLT9	0.792	0.878	0.225	0.500	0.985
Simple RNN	LTT0	0.802	0.892	0.199	0.608	0.999
	LTT1	0.799	0.852	0.195	0.580	0.996
	LTT2	0.929	0.999	0.132	0.938	1.000
	LTT3	0.943	0.999	0.120	0.944	1.000
	LTT4	0.777	0.853	0.203	0.559	0.978
	LTT5	0.745	0.745	0.227	0.504	1.000
	LTT6	0.919	1.000	0.156	0.949	1.000
	LTT7	0.924	0.997	0.148	0.946	1.000
	LTT8	0.921	0.997	0.150	0.947	1.000
	LTT9	0.785	0.895	0.231	0.534	1.000
	PLT0	0.872	0.926	0.181	0.876	0.994
	PLT1	0.858	0.933	0.170	0.782	0.982
	PLT2	0.829	0.898	0.182	0.687	0.990
	PLT3	0.851	0.969	0.180	0.700	0.999
	PLT4	0.848	0.915	0.173	0.738	0.989
	PLT5	1.000	1.000	0.001	1.000	1.000
	PLT6	0.999	1.000	0.003	1.000	1.000
	PLT7	0.850	0.940	0.189	0.712	0.998
	PLT8	0.901	0.964	0.133	0.871	0.996
	PLT9	1.000	1.000	0.000	1.000	1.000

Table 37: Statistics for Brier Score by pattern type and network architecture

Architecture	Pattern	Mean	Median	SD	Q1	Q3
CNN	LTT0	0.209	0.078	0.226	0.000	0.455
	LTT1	0.214	0.090	0.227	0.003	0.491
	LTT2	0.106	0.000	0.187	0.000	0.102
	LTT3	0.078	0.000	0.146	0.000	0.055
	LTT4	0.209	0.068	0.227	0.000	0.455
	LTT5	0.221	0.135	0.230	0.000	0.459
	LTT6	0.052	0.000	0.127	0.000	0.022
	LTT7	0.072	0.001	0.140	0.000	0.040
	LTT8	0.070	0.002	0.137	0.000	0.038
	LTT9	0.222	0.154	0.231	0.000	0.456
	PLT0	0.154	0.010	0.220	0.000	0.389
	PLT1	0.154	0.009	0.221	0.000	0.400
	PLT2	0.136	0.007	0.204	0.000	0.295
	PLT3	0.071	0.004	0.152	0.001	0.018
	PLT4	0.167	0.052	0.206	0.000	0.369
	PLT5	0.000	0.000	0.001	0.000	0.000
	PLT6	0.001	0.000	0.003	0.000	0.000
	PLT7	0.175	0.087	0.206	0.000	0.370
	PLT8	0.147	0.093	0.173	0.002	0.237
	PLT9	0.000	0.000	0.000	0.000	0.000
LSTM	LTT0	0.206	0.107	0.220	0.000	0.465
	LTT1	0.213	0.067	0.227	0.004	0.484
	LTT2	0.140	0.008	0.189	0.000	0.307
	LTT3	0.107	0.068	0.139	0.002	0.143
	LTT4	0.229	0.184	0.218	0.000	0.456
	LTT5	0.244	0.295	0.234	0.000	0.485
	LTT6	0.088	0.001	0.153	0.000	0.072
	LTT7	0.087	0.033	0.131	0.001	0.142
	LTT8	0.117	0.035	0.150	0.001	0.232
	LTT9	0.233	0.163	0.236	0.000	0.479
	PLT0	0.150	0.019	0.203	0.003	0.339
	PLT1	0.194	0.080	0.224	0.001	0.462
	PLT2	0.150	0.017	0.199	0.002	0.297
	PLT3	0.131	0.067	0.157	0.004	0.188
	PLT4	0.210	0.129	0.210	0.003	0.405
	PLT5	0.007	0.002	0.013	0.000	0.004
	PLT6	0.028	0.012	0.033	0.003	0.046
	PLT7	0.203	0.118	0.217	0.001	0.404
	PLT8	0.208	0.245	0.188	0.022	0.321
	PLT9	0.144	0.175	0.109	0.015	0.250
Simple RNN	LTT0	0.208	0.126	0.199	0.002	0.418
	LTT1	0.216	0.175	0.197	0.010	0.419
	LTT2	0.076	0.002	0.136	0.000	0.070
	LTT3	0.071	0.003	0.132	0.000	0.065
	LTT4	0.231	0.152	0.203	0.027	0.442
	LTT5	0.221	0.249	0.200	0.000	0.416
	LTT6	0.082	0.000	0.157	0.000	0.053
	LTT7	0.078	0.003	0.150	0.000	0.055
	LTT8	0.083	0.003	0.154	0.000	0.059
	LTT9	0.220	0.125	0.232	0.000	0.468
	PLT0	0.135	0.088	0.178	0.009	0.141
	PLT1	0.157	0.098	0.168	0.021	0.242
	PLT2	0.184	0.119	0.185	0.019	0.337
	PLT3	0.165	0.073	0.184	0.003	0.327
	PLT4	0.169	0.111	0.178	0.018	0.302
	PLT5	0.000	0.000	0.001	0.000	0.000
	PLT6	0.001	0.000	0.003	0.000	0.000
	PLT7	0.163	0.075	0.194	0.004	0.320
	PLT8	0.118	0.069	0.140	0.010	0.176
	PLT9	0.000	0.000	0.000	0.000	0.000